

GALALA UNIVERSITY

Powered by

Arizona State University®

Faculty of Computer Science and Engineering

Smart gym assistant - Train mate

Ahmed Sameh	222100081
Adham Allam	222100194
Amira Yasser	222100305
Ibrahim Mohamed	A20000011
Ahmed Wael	B20000006

Supervised By

Associate Professor: Mohamed Ghetas

June 2026

Galala University

Faculty of Computer Science & Engineering

Graduation Project

Smart gym assistant - Train mate

A Project submitted to the Faculty of Computer Science and Engineering, Galala University, in partial fulfillment of the requirements for the Bachelor Degree in CS/AIS/AIE

Submitted By

Student Name	Student ID
Ahmed Sameh	222100081
Adham Allam	222100194
Amira Yasser Ahmed	222100305
Ibrahim Mohamed Elgaaly	A20000011
Ahmed Wael Tayea	B20000006

Project Supervisor

Prof. **Mohamed Ghetas**

Approved by the discussion committee:

		Signature
Committee Chair ..... Faculty of Computer Science and Engineering Galala University.	**Chair**	
Committee Member 1 ..... Faculty of Computer Science and Engineering Galala University.	**Member**	
Committee Member 2 ..... Faculty of Computer Science and Engineering Galala University.	**Member**	

Cairo

June – 2026

Graduation Project Declaration Form

Project Title: Smart gym assistant - Train mate
Academic Year: 2025/2026 Program: CS/AIS/AIE
Supervisor: Prof. Mohamed Ghetas

Student Declaration

We, the undersigned students, hereby declare that:

1. This graduation project report and all submitted deliverables are our own original work, except where proper acknowledgment and references have been provided.
2. We have complied with the university regulations regarding academic integrity, plagiarism, copyright, and ethical conduct.
3. All external sources, including books, articles, websites, software libraries, datasets, and AI assisted tools, have been appropriately cited and referenced.
4. We understand that plagiarism, fabrication, falsification of data, or unauthorized copying of work may result in disciplinary action according to university regulations.
5. Any AI tools used during the project served only as supporting tools. We remain fully responsible for the accuracy, originality, implementation, results, and conclusions presented in this project.
6. We certify that the submitted software, designs, models, datasets, experiments, results, and documentation accurately reflect the work performed by our team.
7. We acknowledge that the university may use similarity-checking and plagiarism-detection tools to verify the originality of our submission.

Intellectual Property and Ownership Declaration

We further acknowledge and agree that:

8. Intellectual property developed as part of this project shall be governed by applicable university policies, agreements, and intellectual property laws.
9. Students shall be recognized as the creators of their original contributions to the project.
10. Supervisors shall receive appropriate recognition for substantial academic or intellectual contributions, including authorship or inventorship where applicable.
11. Galala University, through supervisors, retains the right to archive, evaluate, and use the project and its deliverables for academic, research, accreditation, and educational purposes, with proper attribution to the authors.
12. Any publication, patent application, commercialization, or public dissemination arising from this project shall appropriately recognize the contributions of students, supervisors, the university, and any external collaborators in accordance with applicable regulations.
13. No team member may represent the project team, make commitments, disclose project related information, submit project materials, or enter into agreements with any external party on behalf of the team without the prior written approval of the team members and the project supervisor.

Individual Contribution Statement

Each team member declares that the percentage shown below reasonably reflects their contribution to the project.

Student Name	Student ID	Contribution (%)	Signature
Ahmed Sameh	222100081	20%	
Adham Allam	222100194	20%	
Amira Yasser Ahmed	222100305	20%	
Ibrahim Mohamed Elgaaly	A20000011	20%	
Ahmed Wael Tayea	B20000006	20%	

Cairo
June - 2026

AI Usage Disclosure (If Applicable)

Please indicate any AI-assisted tools used during the project:

Tool name	Purpose of use
OpenAI-ChatGPT, Claude. Ai	The AI was used to improve writing and check grammar, simplify technical explanations, assist with code debugging, organize reports, support presentation building, and format documentation. It also helped refine code-related explanations and troubleshoot errors

Examples of acceptable use may include language improvement and proofreading, code debugging assistance, literature search support, and documentation formatting.

Students remain fully responsible for verifying the accuracy, originality, and validity of all generated content.

Supervisor Verification

I have reviewed the submitted graduation project and confirm that, to the best of my knowledge, it has been conducted under my supervision and satisfies the academic requirements of the program.

Supervisor Name: **Prof. Mohamed Ghetas**

Signature: _____ Date: / /

Student Signatures

By signing below, we confirm that we have read and understood this declaration and agree to its contents.

Student Name	Student ID	Signature
Ahmed Sameh	222100081	
Adham Allam	222100194	
Amira Yasser Ahmed	222100305	
Ibrahim Mohamed Elgaaly	A20000011	
Ahmed Wael Tayea	B20000006	

Date: / /

For Committee Use Only

- Similarity Index: _____ %
- Declaration Accepted: □ Yes □ No
- Comments:

.....

.....

.....

Committee Chair Signature: _____ Date: / /

Cairo

June - 2026

Abstract

Physical training without professional supervision often leads to incorrect posture, inefficient repetitions, and a higher risk of injury. Most existing mobile fitness applications are passive: they show pre-recorded videos or static instructions but cannot watch the user, count repetitions, or correct form in real time. This project addresses that gap by designing and building TrainMate, a complete AI-based mobile smart gym assistant, TrainMate is a full-stack system composed of three integrated pillars. The first is a Flutter mobile application that runs on Android and provides authentication, onboarding, live camera-based exercise tracking, workout-plan management, an AI fitness chatbot, body-progress analysis, and a fully bilingual (English/Arabic) user interface. The second is a FastAPI REST backend that provides secure JWT authentication with email verification, user profiles, workout-plan lifecycle management, workout-session storage, AI session reports, and machine-learning endpoints. The third is an artificial-intelligence layer that performs human pose estimation, temporal exercise classification with a Bidirectional Long Short-Term Memory (Bi-LSTM) network, rule-based repetition counting and form-fault detection, and category-aware before/after body-progress comparison, The exercise recognition model converts each video frame into a compact 22-dimensional biomechanical feature vector and analyses a sliding window of thirty consecutive frames. The model recognizes four exercises end-to-end: squat, push-up, shoulder press, and barbell biceps curl. The pipeline is designed for on-device inference through TensorFlow Lite; the build evaluated in this report performs classification through the backend server fallback, with the on-device TFLite path ready to activate once the exported model asset is bundled into the application, This report documents the full engineering process: the literature survey and comparative analysis, the system analysis and methodology, the detailed implementation of the mobile, backend, and AI layers, and a dedicated Results chapter followed by a Discussion chapter. The Results chapter distinguishes explicitly between outcomes that were verified directly on the running system and behaviour that is expected by design and defined as a test protocol for controlled trials. The verified results confirm that TrainMate delivers a working, interactive, full-stack training assistant on commodity mobile hardware while remaining modular and extensible.

Acknowledgements

بسم الله الرحمن الرحيم

﴿وَقُل رَّبِّ زِدْنِي عِلْمًا﴾

نحمد الله عز وجل على ما أنعم به علينّا من علم وقدرة، فما كان هذا العمل ليتم لولا توفيقه وعونه.

We would like to express our sincere gratitude to our supervisor, Associate Professor Mohamed Ghetas, for his continuous guidance, technical insight, and constructive feedback throughout both semesters of this graduation project. His support shaped the scientific and engineering quality of this work, we are grateful to the Faculty of Computer Science and Engineering at Galala University for providing the academic environment, tools, and encouragement that made this project possible, and finally we thank the Graduation Project Committee for the detailed first-semester feedback. Their comments on literature depth, citations, scientific rigour, and report structure directly guided the improvements presented in this final report.

Date: / /

Table of Contents

Abstract 5
Acknowledgements 6
Table of Contents 7
List of Figures 10
1 Introduction 14
1.1 Overview 14
1.2 Problem Statement 15
1.3 Motivation 16
1.4 Project Objectives 16
1.5 Project Contributions 17
1.6 Scope of Work 18
1.7 Work Plan 18
2 Background and Literature Survey 20
2.1 Background Concepts 20
2.1.1 Human Pose Estimation 21
2.1.2 Feature Engineering for Movement 21
2.1.3 Temporal Modeling with Bi-LSTM 21
2.2 Related Work 22
2.2.1 MediaPipe-Based Posture Estimation (Biswas et al., 2023) [2] 22
2.2.2 OpenPose + Deep Learning Action Recognition (Chen et al., 2024) [5] 22
2.2.3 ML-Based Automated Gym Trainer (Sharma et al., 2023) [3] 22
2.2.4 Computer Vision in Interactive Fitness — Review (Al-Sayed & Hassan, 2025) [7] 23
2.2.5 MediaPipe Fitness Tracker Prototype (2024) 23
2.2.6 Fit Track: Predictive Gym Analytics (2023) 23
2.3 Comparative Study and Research Gap 23
3 System Analysis 26
3.1 Solution Methodology 26
3.1.1 Stage 1 — Pose Estimation 27
3.1.2 Stage 2 — Feature Extraction 27

3.1.3 Stage 3 — Temporal Classification 28
3.1.4 Stage 4 — Repetition Counting and Form Analysis 29
3.1.5 Stage 5 — Feedback, Persistence, and Coaching 30
3.2 System Architecture 30
3.3 Data Flow and Database Design 31
3.4 Functional and Non-Functional Requirements 35
4 Implementation 39
4.1 Mobile Application (Frontend) 39
4.2 Backend (FastAPI REST API) 40
4.3 Artificial Intelligence Layer 43
4.3.1 Body-Progress Analysis Pipeline 44
4.4 Application Screens (Real Captures) 45
4.5 Implementation Challenges 60
5 Results 62
5.1 Evaluation Approach and Metrics 62
5.2 Experimental Setup 62
5.3 Classifier Configuration and Deployment 63
5.4 Verified Results from the Live System 64
5.5 Repetition Counting and Form Analysis — Test Protocol 65
5.6 Latency and Responsiveness (Targets) 65
5.7 Body-Progress Analysis — Controlled Trials 67
5.8 Case Studies 70
5.8.1 Case Study A — Authenticated Session and Active Plan (verified) 70
5.8.2 Case Study B — Live Tracking Launch and Server Inference (verified) 70
5.8.3 Case Study C — Chatbot Coaching (verified) 71
5.8.4 Case Study D — Body-Progress Controlled Trials (verified) 71
5.9 Verification and Validation Summary 71
5.10 Overview of AI Components in TrainMate 72
5.11 Supervised Learning — Bi-LSTM Exercise Classifier 73
5.11.1 2.1 How the classifier was learned 73
5.11.2 Training and validation curves (accuracy & loss) 74
5.11.3 Per-class validation metrics 76

5.11.4 Model deployment (export) 76
5.12 Body-Progress Analyzer — Validation (No ML Training) 76
5.13 Rep-Counter FSM — Live Validation (No ML Training) 77
5.14 AI Chatbot — Pre-Trained LLM (No Project Training) 78
6 Discussion 80
6.1 Interpretation of Results 80
6.2 Significance and Impact 81
6.3 Limitations 82
6.4 Alignment with Objectives and Future Work 83
7 References 85

List of Figures

Figure 1: TrainMate mobile application user flow 15
Figure 2: Real-time exercise recognition pipeline 26
Figure 3: 22-dimensional frame feature vector composition 28
Figure 4: Bidirectional LSTM classifier architecture 28
Figure 5: Repetition counting state machine 29
Figure6 : Three-tier system architecture 30
Figure 7: End-to-end data flow 31
Figure 8: Backend use-case diagram 32
Figure 9: Database ORM class diagram 33
Figure 10: Authentication sequence diagram 34
Figure 11: Workout save and chatbot sequence 42
Figure 12: Login screen — JWT authentication entry point 46
Figure 13: Smart home screen — active plan and weekly flow 47
Figure 14: Workout hub — prescription and exercise catalog 48
Figure 15: Exercise setup — options before the session 49
Figure 16: Coaching phase — pre-session form tips 50
Figure 17: Live exercise tracking — camera with HUD 51
Figure 18: Live HUD statistics — detailed session state 52
Figure 19: Create plan — templates and exercise selection 53
Figure 20: Plan settings — active plan management 54
Figure 21: Profile dashboard — performance summary 55
Figure 22: Settings hub 56
Figure 23: App settings — backend status, theme, language 57
Figure 24: Arabic (RTL) interface — full bilingual support 58
Figure 25: AI chatbot — contextual fitness coaching 59
Figure 26: Verified capabilities and model configuration 63
Figure 27: Verified workout session results from physical device 66
Figure 28: Body-progress Trial A — genuine before/after change (Muscle Gain) 68
Figure 29: Body-progress Trial B — identical photos correctly report no measurable change 69
Figure 30: Body-progress Trial C — black after photo rejected by pose guard 69

Figure 31: Body-progress Trial D — unsuitable after photo rejected by pose guard 70
Figure 32: AI pipeline validation overview (all modules) 73
Figure 33: Training dataset class balance 74
Figure 34: Training vs validation loss (110 epochs) 74
Figure 35: Training vs validation accuracy (110 epochs) 75
Figure 36: Confusion matrix on validation split 75
Figure 37: Body-progress validation outcomes 77
Figure 38: Rep-counter live validation by exercise 77
Figure 39: Session-level rep counts (physical device) 78

List of Tables

Table 1: Project work plan (October 2025 – June 2026) 18
Table 2: Comparative study of related systems 23
Table 3: Selected pose landmarks 27
Table 4:Functional requirements 36
Table 5: Non-functional requirements 37
Table 6: Flutter packages and roles 39
Table 7: Backend REST API endpoints 40
Table 8: Database entities 41
Table 9: Form-fault codes 44
Table 10: Body-progress physique metrics and category use 45
Table 11: Evaluation metrics and how each is obtained 62
Table 12: Bi-LSTM configuration (verified against model_training.py) 63
Table 13: Capabilities verified on the running system 64
Table 14: Repetition counting and form test protocol (results recorded during physical-device trials) 65
Table 15: Responsiveness targets and measurement plan 66
Table 16: Exercise session aggregates (verified from workout_sessions table) 66
Table 17: Body-progress controlled trials (verified on evaluation account) 67
Table 18: Verification and validation summary 71
Table 19: AI components — training vs validation 72
Table 20: Bi-LSTM validation report 76
Table 21: Body-progress controlled trials 76
Table 24: Objective fulfilment 84

Introduction

1 Introduction

1.1 Overview

The fitness and health industry has been transformed by advances in artificial intelligence (AI) and computer vision. Smartphones now carry enough processing power to run pose-estimation and deep-learning models locally, which makes it possible to deliver personal-trainer-style feedback to anyone with a phone and a camera. Despite this, the majority of commercial fitness applications remain fundamentally passive: they present instructional videos, timers, and manual logging screens, but they do not observe the user during exercise execution and cannot react to mistakes as they happen [1], [7], trainMate was designed to close this gap. It is an AI-based mobile smart gym assistant that uses the phone camera to estimate the user's body pose, recognizes which exercise is being performed, counts repetitions, evaluates posture quality, and provides immediate corrective feedback. Beyond the live training loop, TrainMate is a complete fitness companion: it manages user accounts, personalized workout plans, workout history, body-progress photo analysis, and an AI chatbot that answers fitness questions using the user's own profile and plan as context, importantly, TrainMate is not a single script or a notebook prototype. It is engineered as a production-shaped, three-tier system. The presentation tier is a Flutter mobile application; the application tier is a FastAPI REST backend backed by a relational database; and the intelligence tier contains the pose-and-feature pipeline, the trained Bidirectional LSTM classifier exported to TensorFlow Lite, and the body-progress analyzer, this separation of concerns is a core engineering contribution of the project, because it allows each layer to be developed, tested, and extended independently, the system currently supports four exercises end-to-end — squat, push-up, shoulder press, and barbell biceps curl — chosen because they cover the major lower-body, upper-body push, overhead, and isolated-arm movement patterns, and because they can be reliably distinguished from a single front-facing camera. Figure 1 summarizes the overall navigation flow of the mobile application, from launch and authentication through onboarding to the live training and supporting feature screens.

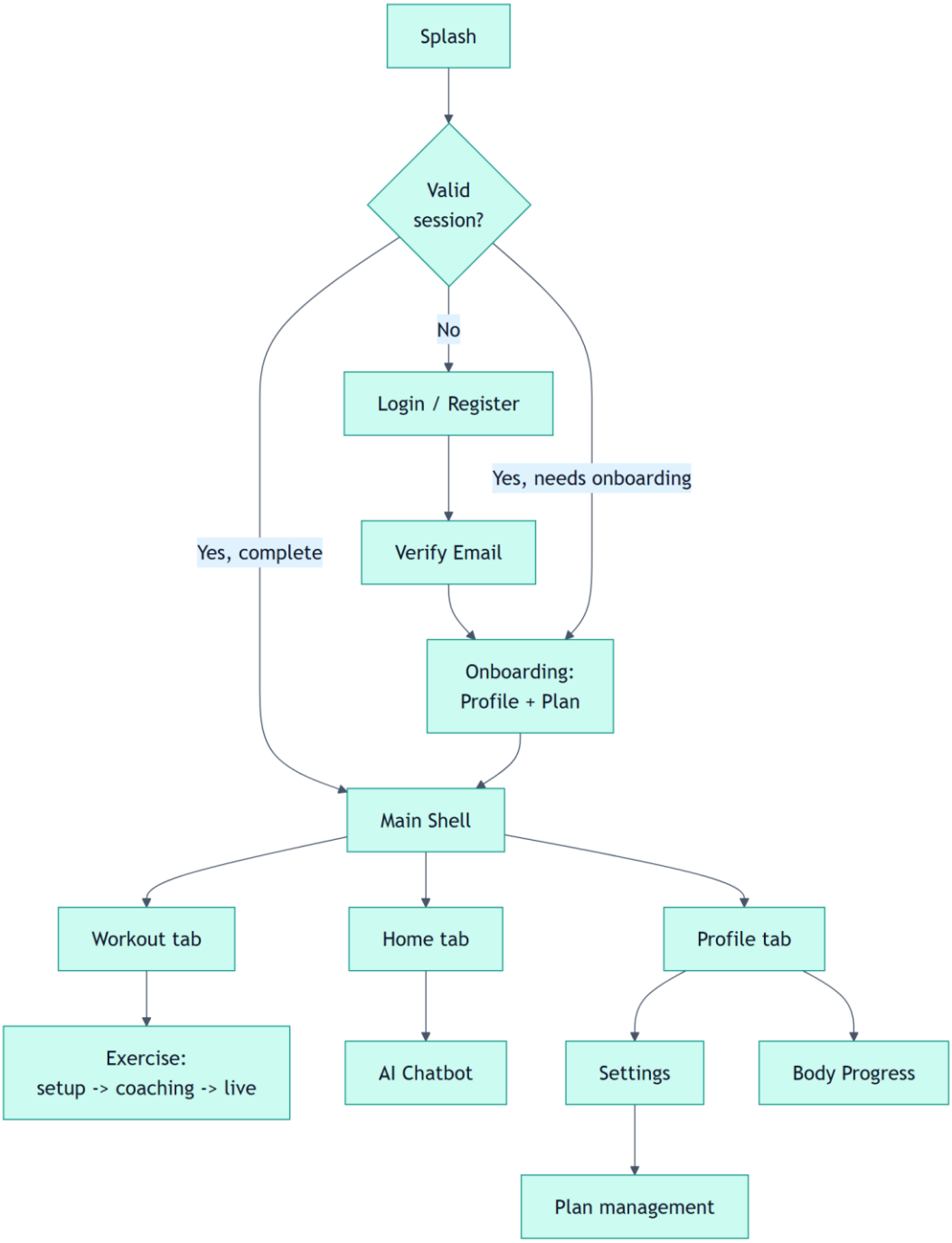


Figure 1: TrainMate mobile application user flow

1.2 Problem Statement

Several concrete problems motivated the development of TrainMate. Each problem is directly addressed by one or more system features described later in this report, incorrect exercise execution. Without expert supervision, users frequently perform squats, push-ups, and overhead presses with poor joint alignment and incomplete range of motion, reducing effectiveness and increasing injury risk [1], [7].

Absence of a real-time feedback loop. Static videos and text cannot detect a shallow squat, a sagging hip during a push-up, or a swinging biceps curl while it is happening.

Unreliable manual repetition counting. Self-counting is inconsistent and never measures the quality of the movement, only (approximately) its quantity, dependence on human trainers. Professional supervision is expensive and not continuously available, which limits access to good coaching, fragmented tools. Users typically need separate apps for logging, coaching, and progress photos, none of which share a common profile or plan context, weak long-term progress measurement. Meaningful improvement requires session history, plan-completion analytics, and objective before/after comparison rather than isolated daily counters.

1.3 Motivation

The primary motivation is accessibility: making professional-quality movement feedback available to ordinary users at no marginal cost, on hardware they already own. A second motivation is injury prevention. Because TrainMate validates joint angles and movement phases against biomechanical thresholds, it can warn users about unsafe patterns before they accumulate into injuries [7], [11], from an academic standpoint, the project is an opportunity to integrate several disciplines into one coherent product: computer vision (pose estimation), deep learning (sequence classification), mobile software engineering (Flutter), distributed systems (REST API design and authentication), and human-computer interaction (bilingual, real-time UI). Building a working end-to-end system, rather than an isolated model, was a deliberate goal.

1.4 Project Objectives

Project objectives state what this graduation project aims to accomplish—they define the intended outcomes and deliverables. They are distinct from the project contributions presented in Section 1.5, which describe the original value this work adds relative to existing literature and systems.

- O1 — To design and implement a smartphone-based system that performs real-time human pose estimation and delivers live exercise assistance during camera-based workout sessions.
- O2 — To develop, train, and validate a Bidirectional LSTM classifier for recognizing squat, push-up, shoulder press, and barbell biceps curl using engineered pose-feature sequences.
- O3 — To implement interpretable, on-device repetition counting and biomechanical form-fault detection that operate during live workouts without requiring cloud inference per frame.

- O4 — To build a secure full-stack backend that supports user authentication, workout-plan management, session history, AI chatbot integration, and before/after body-progress analysis.
- O5 — To deliver a bilingual (English/Arabic) mobile user interface and verify its usability across the primary application workflows.
- O6 — To evaluate the integrated system using defined performance metrics, controlled experiments, functional verification, and documented, reproducible test protocols.

1.5 Project Contributions

Project contributions describe the original knowledge, methods, and engineering value that this work introduces beyond what prior systems provide. Whereas objectives specify what the team set out to achieve, contributions explain what is new—relative to the research gap identified in Section 2.11—and why the deliverable matters academically and practically.

- **C1 — System integration contribution:** A deployable three-tier mobile–cloud architecture that combines on-device pose analysis, temporal exercise recognition, secure backend persistence, category-aware body-progress analysis, and context-grounded chatbot coaching in one coherent product. Prior works such as Biswas et al. [1] and Chen et al. [5] address isolated algorithmic tasks; this project contributes a complete, end-to-end system design (Figure 2) that closes the integration gap summarized in Table 2.
- **C2 — Methodological contribution:** A hybrid recognition pipeline that pairs a compact 22-dimensional scale-invariant pose feature vector with a Bidirectional LSTM temporal classifier, and couples it with explicit finite-state-machine repetition counting and rule-based form-fault codes (Table 8). This design balances learned temporal discrimination with interpretable biomechanical feedback—a combination not present as an integrated methodology in the reviewed literature [3], [5], [7].
- **C3 — Technical contribution:** A category-weighted body-progress analyzer with pose-guard validation that (a) rejects invalid after photos before meaningless scores are produced, and (b) reports neutral baseline scores when identical before/after images are submitted (Trial B, Table 14). This contributes a reproducible validation protocol for honest progress measurement rather than inflated improvement claims.
- **C4 — Engineering and reproducibility contribution:** A documented, reproducible implementation spanning Flutter, FastAPI, and TensorFlow/TFLite export paths (REPRODUCIBILITY.md [49]), with verified live evaluation on physical hardware—ten

persisted workout sessions totalling 94 repetitions (Table 11)—and explicit separation of confirmed results from future measurement protocols.

- **C5 — Human–computer interaction contribution:** A bilingual (English/Arabic, RTL) fitness-assistant design with per-user chat privacy, real-time HUD coaching, and localized session reports, demonstrating practical accessibility for Egyptian users without sacrificing the technical rigour expected in a computer-science graduation project.

1.6 Scope of Work

The project scope covers the complete TrainMate system. In-scope items include: the Flutter Android application and all of its screens; the FastAPI backend with its database, authentication, and AI integration; the machine-learning pipeline for the four supported exercises; the body-progress analyzer; and the AI chatbot integration through the backend, out-of-scope items, documented honestly as boundaries, include: exercises beyond the four supported movements; multi-person simultaneous tracking; wearable-sensor fusion; and clinical or medical diagnosis. These boundaries are revisited as limitations and future work in Chapter 6.

1.7 Work Plan

The project was executed across two academic semesters. The first semester focused on research, design, model development, and the initial report; the second semester focused on full-stack implementation, integration, evaluation, and this final report. Table 1 summarizes the plan.

Table 1: Project work plan (October 2025 – June 2026)

Phase	Duration	Main Deliverables
Literature review and gap analysis	4 weeks	Related-work matrix, requirements draft
System design and API specification	4 weeks	Architecture, DB schema, UI flows
Dataset preprocessing and model training	6 weeks	Bi-LSTM model, scaler, encoder, TFLite export
Mobile app and backend implementation	6 weeks	Auth, plans, exercise engine, chatbot, body AI
Integration, testing, optimization	4 weeks	Functional tests, latency tuning, bilingual QA
Final report and presentation	2 weeks	Results + Discussion chapters, live demo

Background and Literature Survey

2 Background and Literature Survey

The successful design of a real-time, smartphone-based exercise monitoring system requires grounding in several interrelated research domains: computer vision and human pose estimation, temporal action recognition, biomechanical posture evaluation, mobile and cloud software architecture, and intelligent user interaction through conversational agents. Chapter 1 established the problem context—that unsupervised training often leads to incorrect form, unreliable repetition counting, and fragmented fitness tools—and defined the project objectives and contributions that guide this work. The purpose of this chapter is to survey the theoretical background and prior systems that inform the proposed solution, identify their strengths and limitations, and justify the technical choices adopted in subsequent chapters, the literature review is organized as follows. Section 2.2 discusses the role of artificial intelligence in fitness applications and the shift from passive content delivery toward interactive, vision-based coaching. Section 2.3 introduces computer vision for human motion analysis, which forms the perceptual foundation of the system. Section 2.4 compares major pose estimation frameworks—OpenPose, MediaPipe Pose, and MoveNet—with emphasis on the accuracy–latency trade-offs relevant to mobile deployment. Sections 2.5 through 2.7 review exercise recognition, repetition counting, and posture evaluation techniques, respectively, highlighting the gap between algorithmic prototypes and deployable products, section 2.8 examines AI chatbots in healthcare and fitness contexts, while Section 2.9 surveys mobile fitness applications and their typical functional boundaries. Section 2.10 presents related work through comparative analysis of representative studies, and Section 2.11 states the research gap that this graduation project addresses. The chapter concludes with a summary linking the surveyed literature to the system analysis presented in Chapter 3.

2.1 Background Concepts

Before reviewing related systems and comparative studies, it is necessary to introduce the core technical concepts on which the proposed system is built. This section outlines the fundamental background in human pose estimation, biomechanical feature engineering, and temporal deep learning that underpin real-time exercise monitoring on a smartphone. These concepts provide the vocabulary and theoretical basis for the design and implementation details presented in Chapters 3 and 4.

2.1.1 Human Pose Estimation

Human pose estimation is the computer-vision task of locating anatomical keypoints (joints) of the human body in an image or video frame. Two families dominate the literature. The first is high-accuracy, multi-person frameworks such as OpenPose, which use part-affinity fields to associate joints across people and produce rich skeletons, at a significant computational cost [2], [6]. The second is lightweight, single-person, real-time frameworks such as Google MediaPipe Pose, which output 33 body landmarks per frame and are explicitly optimized for mobile devices [8], trainMate adopts the lightweight philosophy. On the mobile device it uses Google ML Kit Pose Detection (built on the MediaPipe pose model) in accurate streaming mode, which yields 33 landmarks per frame in real time. This choice is central to the project: it makes on-device, low-latency analysis feasible without a server round-trip for every frame.

2.1.2 Feature Engineering for Movement

Raw landmark coordinates are not, by themselves, a good representation of movement. They depend on the user's distance from the camera, the user's body proportions, and the camera angle. To obtain a scale-invariant and position-invariant representation, the raw landmarks are transformed into biomechanical features: joint angles (which are invariant to scale and translation), normalized inter-joint distances, and vertical displacement features [11]. An elbow flexion of 90 degrees means the same thing regardless of whether the user is tall or short, near or far, trainMate encodes each frame as a 22-dimensional feature vector built from these three groups. This compact representation is the input to the temporal classifier and is computed identically in the Python training pipeline and in the Dart on-device pipeline, guaranteeing train/inference consistency.

2.1.3 Temporal Modeling with Bi-LSTM

Exercises are temporal processes, not static poses. A single frame cannot distinguish the bottom of a push-up from a static plank, nor a descending squat from an ascending one. Recurrent neural networks, and in particular Long Short-Term Memory (LSTM) networks, are designed to model sequential dependencies and avoid the vanishing-gradient problem of vanilla recurrent networks [19]. A Bidirectional LSTM processes the sequence in both forward and backward directions, giving each timestep access to past and future context, which is valuable for recognizing cyclic, repetitive movements [10], trainMate's classifier is a stacked Bidirectional LSTM that consumes a

window of 30 consecutive 22-dimensional feature vectors and outputs a probability distribution over the four supported exercises.

2.2 Related Work

This section reviews representative prior systems most relevant to the proposed mobile exercise-monitoring platform. Each study is summarized in terms of its core contribution, primary limitation, and relevance to the objectives defined in Chapter 1. A comparative analysis (Table 2) follows, highlighting the integration gap that the present project addresses.

2.2.1 MediaPipe-Based Posture Estimation (Biswas et al., 2023) [2]

Biswas et al. present a real-time posture-estimation pipeline for AI gym training using MediaPipe and OpenCV. The work demonstrates that lightweight vision pipelines can detect incorrect postures with low latency on commodity hardware. Critically, however, it stops at posture estimation: it does not provide sequence-based exercise classification, a mobile product, plan management, or persistence. TrainMate builds on the feasibility this work establishes and extends it into a complete, deployable application with temporal recognition and a backend.

2.2.2 OpenPose + Deep Learning Action Recognition (Chen et al., 2024) [5]

Chen et al. combine OpenPose skeletons with CNN and LSTM models for human action recognition, achieving strong temporal accuracy. The trade-off is computational cost: OpenPose and deep CNNs are heavy for real-time mobile use. TrainMate deliberately favours deployability by using compact 22-dimensional engineered features and on-device TFLite inference, accepting a smaller model in exchange for real-time performance on a phone.

2.2.3 ML-Based Automated Gym Trainer (Sharma et al., 2023) [3]

Sharma and Patel automate exercise detection and repetition counting using handcrafted features and classical machine learning. The system is practical but its handcrafted, per-exercise features limit scalability to new movements. TrainMate combines a learned temporal classifier (which captures movement dynamics) with explicit, interpretable angle-based rules for repetition counting and form faults, balancing learnability with transparency.

2.2.4 Computer Vision in Interactive Fitness — Review (Al-Sayed & Hassan, 2025) [7]

This review surveys form-analysis techniques and emphasizes injury prevention, dataset limitations, and deployment constraints. It is strong theoretically but offers no complete implementation or benchmark. TrainMate operationalizes the review's recommendations: it implements a concrete set of measurable form-fault codes and reports their behaviour in the Results chapter.

2.2.5 MediaPipe Fitness Tracker Prototype (2024)

A practical, mobile-oriented tracker that uses MediaPipe with rule-based feedback. Its main limitations are a narrow exercise set and the absence of backend persistence or accounts. TrainMate adds secure authentication, workout plans, session history, an AI chatbot, and body-progress scoring on top of the live tracking idea.

2.2.6 Fit Track: Predictive Gym Analytics (2023)

Fit Track focuses on long-term member analytics and dashboards rather than live pose correction. It highlights the value of progress tracking. TrainMate complements live coaching with exactly this kind of longitudinal data: it stores every session and computes plan-completion metrics and AI body-progress scores.

2.3 Comparative Study and Research Gap

Table 2 contrasts the reviewed systems against TrainMate along the dimensions most relevant to a deployable smart gym assistant.

Table 2: Comparative study of related systems

#	System	Strength	Limitation	TrainMate's Response
1	Biswas et al. [2]	Real-time posture estimation	No full product / no recognition	Flutter app + Bi-LSTM recognition
2	Chen et al. [5]	High temporal accuracy	Heavy, not mobile-friendly	Compact features + on-device TFLite
3	Sharma et al. [3]	Automated counting	Handcrafted, low scalability	Learned model + interpretable rules

4	Al-Sayed et al. [7]	Strong form theory	No implementation/benchmark	Concrete form-fault engine + results
5	MediaPipe tracker (2024)	Practical mobile demo	Few exercises, no backend	Full REST backend + accounts
6	Fit Track (2023)	Longitudinal analytics	No live CV correction	Live coaching + body-progress AI

The comparison exposes a clear research and engineering gap. Existing systems each optimize a single dimension — pose accuracy, mobility, analytics, or feedback — but none integrate all of them into one coherent, deployable, and scientifically evaluated product within the scope of a graduation project, trainMate is positioned precisely in this gap. It unifies (1) on-device real-time recognition, (2) interpretable repetition and form analysis, (3) a complete backend with authentication and plan lifecycle, (4) AI chatbot coaching, (5) body-progress analysis, and (6) bilingual UX — and it backs these claims with the experimental evaluation presented in Chapter 5.

System Analysis

3 System Analysis

Following the literature survey in Chapter 2, this chapter translates the identified research gap and project objectives into concrete system requirements and design artifacts. It specifies the functional and non-functional requirements of the proposed mobile computer-vision system, defines stakeholder roles, and presents the three-tier architecture that integrates the Flutter client, FastAPI backend, and AI processing layer. UML diagrams—including use-case, activity, sequence, and class views—are provided to model user interactions, data flows, and database structure. The chapter also documents the REST API groups and logical database design that support authentication, workout management, body-progress analysis, and chatbot integration, together, these elements form the blueprint for the implementation described in Chapter 4.

3.1 Solution Methodology

The core of TrainMate is the real-time exercise-analysis pipeline shown in Figure 2. Each camera frame flows through a sequence of stages that progressively transform raw pixels into actionable coaching feedback. The stages are described in detail below.

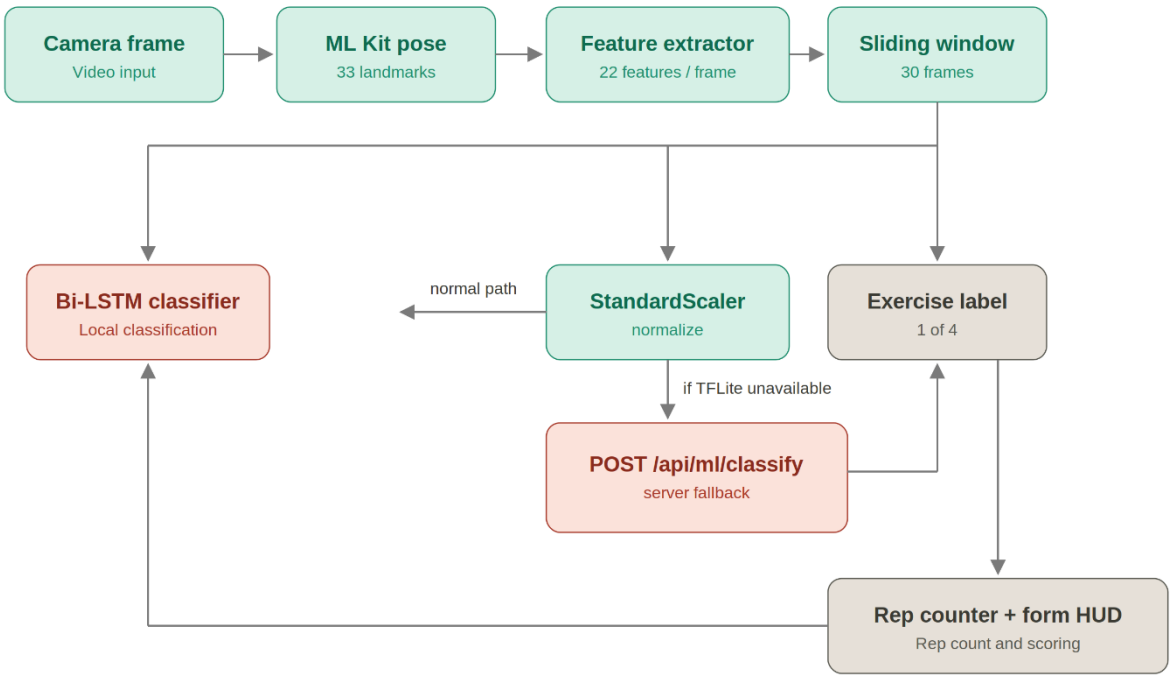


Figure 2: Real-time exercise recognition pipeline

3.1.1 Stage 1 — Pose Estimation

Each frame from the live camera is passed to Google ML Kit Pose Detection in accurate streaming mode. The detector returns up to 33 body landmarks, each with image-plane coordinates and a visibility likelihood. From these, TrainMate selects the twelve joints that dominate the supported exercises; the selection is listed in Table 3. Concentrating on these joints reduces noise and computation while retaining the information needed for angle and distance features.

Table 3: Selected pose landmarks

Joint group	Landmarks (MediaPipe indices)
Shoulders	left 11, right 12
Elbows	left 13, right 14
Wrists	left 15, right 16
Hips	left 23, right 24
Knees	left 25, right 26
Ankles	left 27, right 28

3.1.2 Stage 2 — Feature Extraction

Each frame is encoded into a 22-dimensional feature vector composed of three groups, illustrated in Figure 3:

Eight joint angles, computed with vector geometry (the dot-product formula) at the elbow, knee, hip, and shoulder on both sides. Angles are inherently scale- and translation-invariant.

Twelve normalized Euclidean distances between key joint pairs, each divided by a stable body-span reference (such as shoulder-to-hip distance) to remove the effect of camera distance and body size.

Two vertical offset features (elbow-to-shoulder vertical distance, left and right), which emphasize the up/down motion that characterizes presses and push-ups.

This feature computation is implemented identically in the Python training pipeline and the Dart runtime extractor, which guarantees that the model sees the same representation during training and inference.

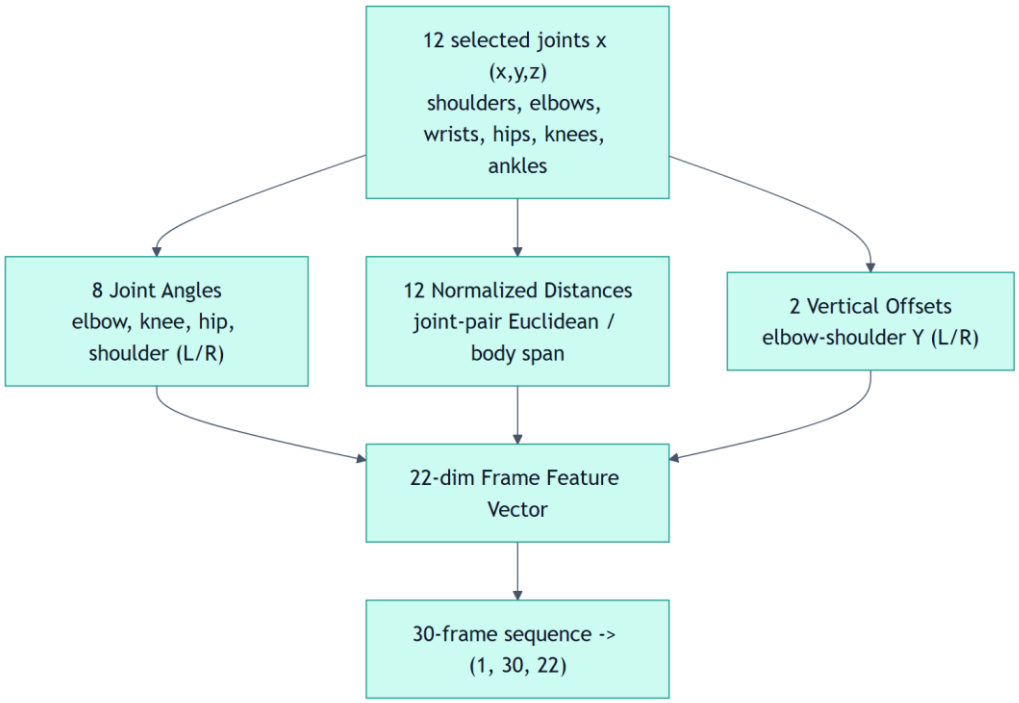


Figure 3: 22-dimensional frame feature vector composition

3.1.3 Stage 3 — Temporal Classification

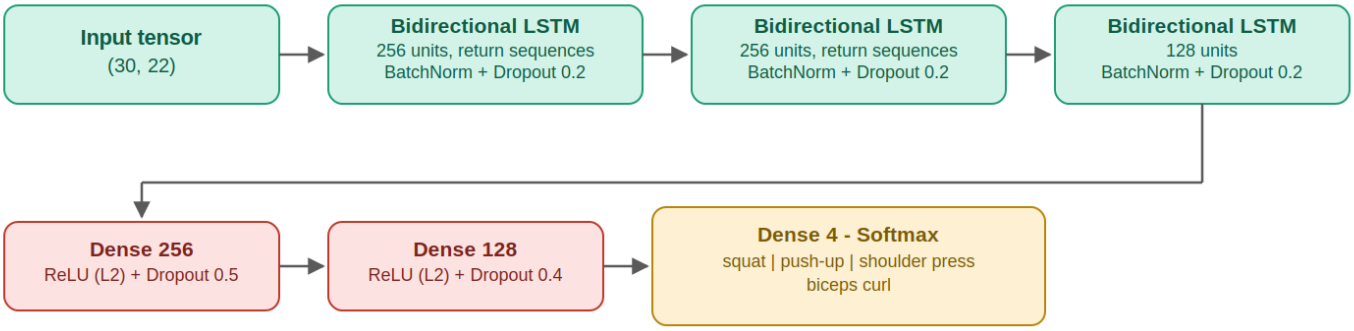


Figure 4: Bidirectional LSTM classifier architecture

Thirty consecutive feature vectors are accumulated into a sliding window, forming a tensor of shape (1, 30, 22). The window is standardized using a StandardScaler fitted during training, then fed to the Bidirectional LSTM classifier shown in Figure 4. The network outputs a softmax probability distribution over the four exercises, and the class with the highest probability becomes the active label. The system implements two interchangeable inference paths against an identical 30×22 input contract: an on-device TensorFlow Lite path, and a backend path through POST /api/ml/classify that runs the equivalent Keras model. In the build evaluated in this report the on-device TFLite asset is not bundled, so classification is served by the backend path; this is visible at runtime as the 'Auto via server' indicator on the live HUD (Figure 18).

3.1.4 Stage 4 — Repetition Counting and Form Analysis

Once the active exercise is known, the repetition counter activates exercise-specific logic implemented as a finite-state machine over joint angles. Each exercise is modelled as a cycle between a flexion (down) phase and an extension (up) phase; a repetition is counted only when a complete, valid transition occurs. This prevents partial movements and landmark noise from inflating the count. For bilateral exercises such as the shoulder press and biceps curl, both limbs must satisfy the movement conditions before a repetition is registered, in parallel, the system evaluates posture against biomechanical thresholds and flags form faults such as a shallow squat, a sagging or piking hip during a push-up, an excessively leaning torso, a swinging curl, or incomplete lockout. The proportion of clean repetitions is tracked as a good-form ratio. Figure 5 shows the state machine and the fault set.

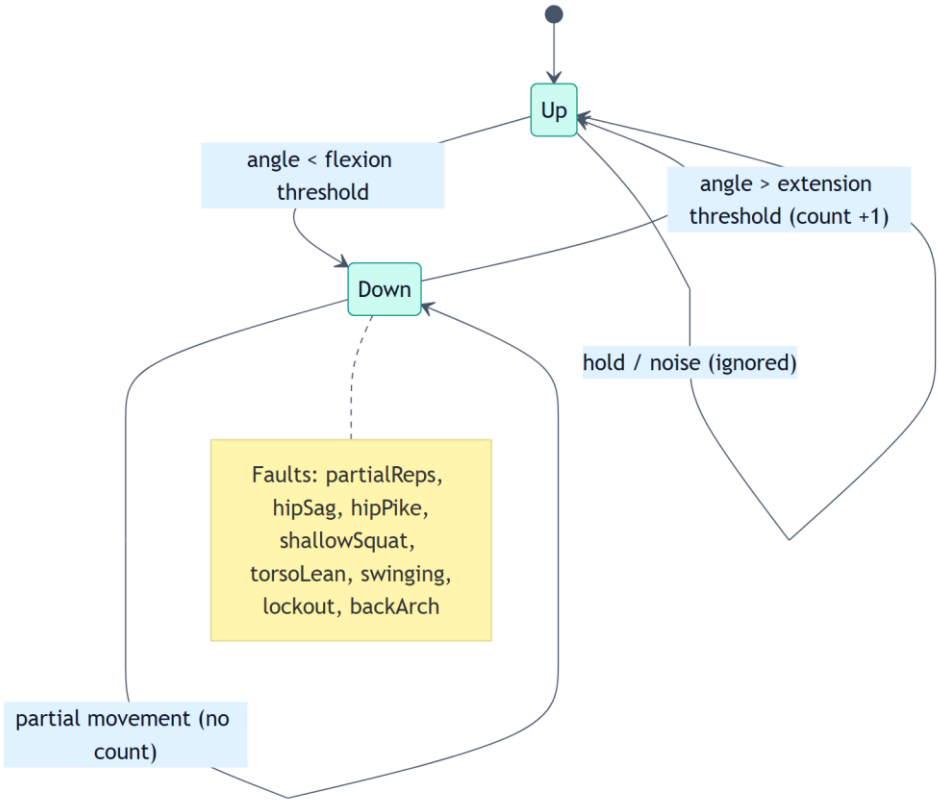


Figure 5: Repetition counting state machine

3.1.5 Stage 5 — Feedback, Persistence, and Coaching

Live feedback is rendered as a heads-up display (HUD) over the camera preview, showing the detected exercise, repetition count, elapsed time, estimated calories, and any active form fault. When the session ends, it is saved through POST /api/workouts together with reps, duration, equipment, sets, and weight; the backend optionally generates a natural-language session report. Aggregated coaching feedback is later surfaced on the home screen through POST /api/chat/progress-feedback.

3.2 System Architecture

TrainMate is organized into three tiers, shown in Figure 6. The separation lets each tier evolve independently and supports the dual on-device/server inference strategy.

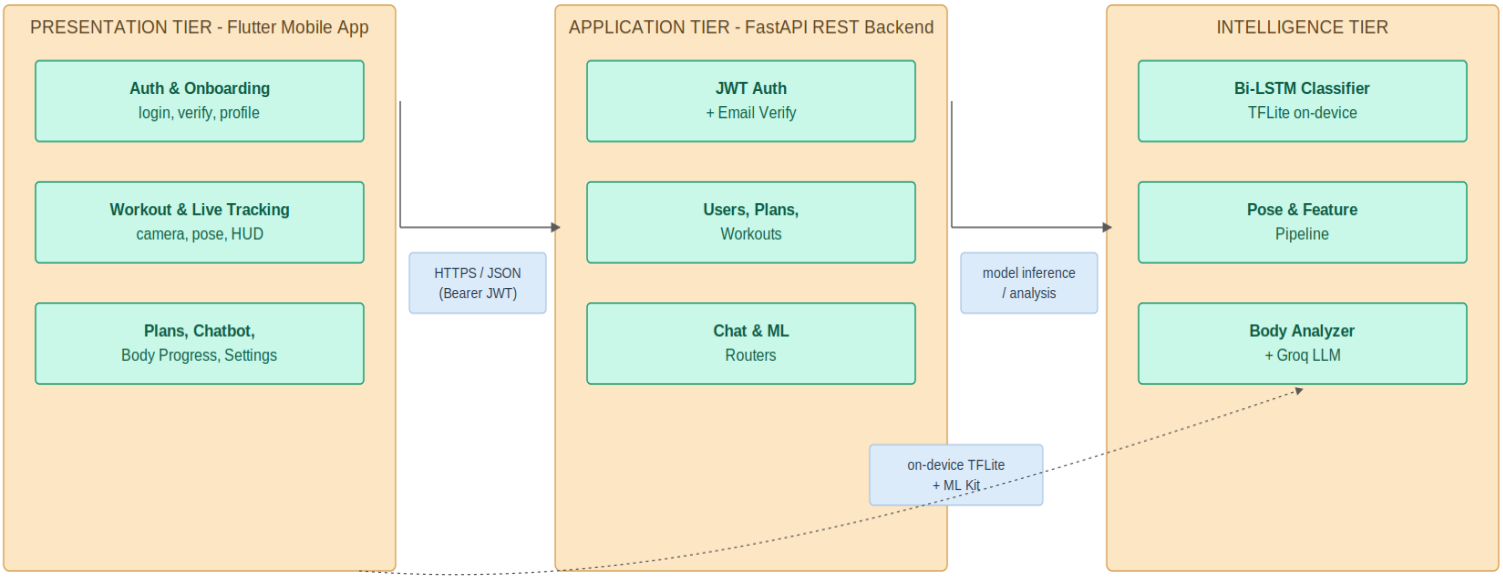


Figure6 : Three-tier system architecture

Presentation tier — a Flutter application providing all user-facing screens, the live camera pipeline, on-device TFLite inference, and the bilingual Material 3 interface.

Application tier — a FastAPI backend exposing REST endpoints for authentication, users, plans, workouts, chat, and machine learning, backed by a relational database accessed through SQLAlchemy.

Intelligence tier — the pose/feature pipeline, the trained Bi-LSTM (Keras for training, TFLite for deployment), and the MediaPipe-based body-progress analyzer, augmented by an external LLM provider for the chatbot and session reports.

3.3 Data Flow and Database Design

Figure 7 shows the end-to-end data flow, and Figures 8–10 present the backend use cases, the database class diagram, and the authentication sequence. The persistent data model is summarized in Table 8 (Chapter 4). The most important entities are User and UserProfile (identity and physical attributes), WorkoutPlan (the multi-plan system holding exercises, photos, target sessions, and AI completion scores), and WorkoutSession (each logged exercise bout).

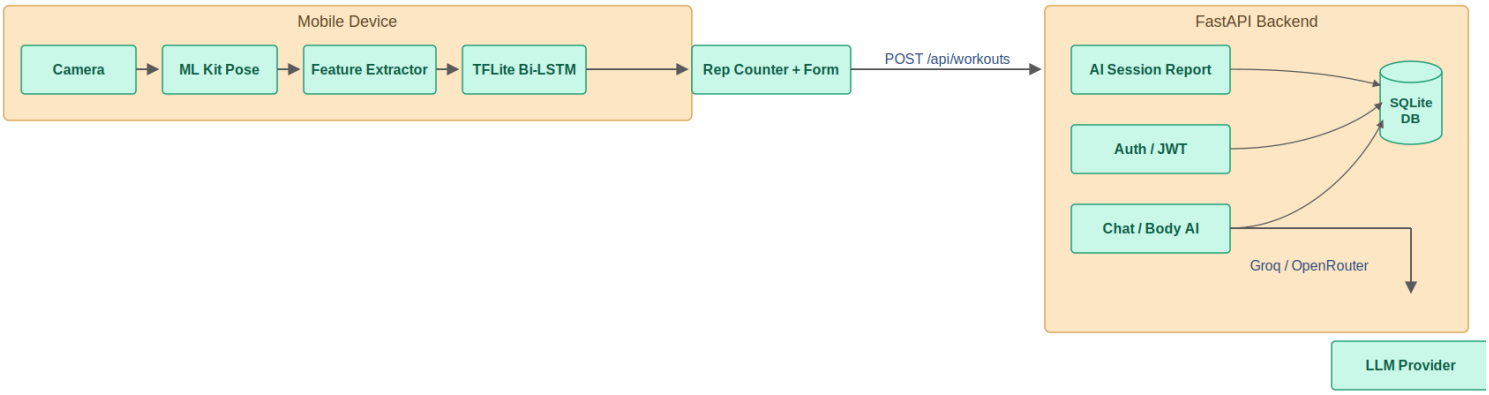


Figure 7: End-to-end data flow

Figure 7 illustrates the complete data path from camera capture to persistent storage and intelligent feedback. The upper subgraph represents the **live workout loop** executed on the mobile device. The camera supplies video frames to **ML Kit Pose**, which extracts skeletal landmarks on each frame. The **22-feature extractor** converts landmarks into scale-invariant biomechanical features. These features feed the **Bi-LSTM classifier** for exercise recognition and the **rep counter with form HUD**, which together deliver real-time coaching output to the user without requiring a server round-trip for every frame, when the user completes a workout, session aggregates—exercise label, repetition count, duration, sets, and related metadata—are transmitted to the backend through **POST /api/workouts**. The lower subgraph shows the **FastAPI backend and SQLite database**. All protected operations pass through **JWT authentication**, ensuring that only verified users can read or write their own records. Saved workouts are stored in the database and may trigger **AI session report** generation, the diagram also shows secondary backend flows. **Chat and body-progress AI** modules read and write user-scoped data in SQLite and communicate with an **external LLM** (Groq or OpenRouter) when natural-language coaching or progress feedback is required. This separation clarifies a key design decision: latency-critical

pose processing remains on-device, while persistence, history, chatbot context, and progress analysis are handled centrally by the backend.

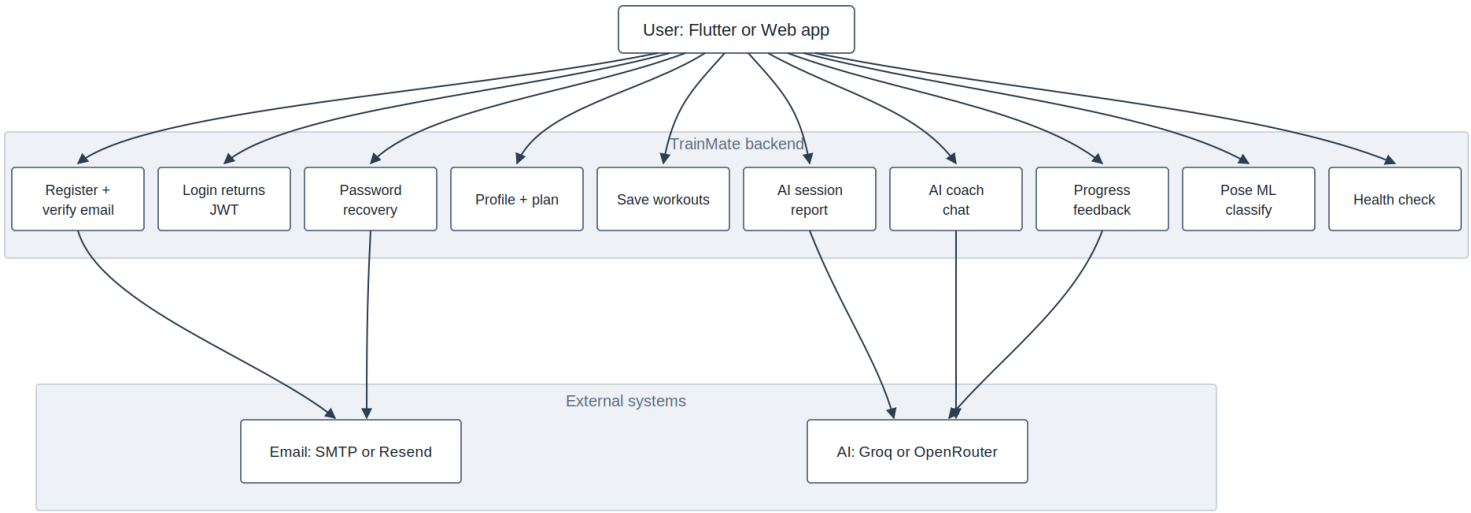


Figure 8: Backend use-case diagram

Figure 8 defines the functional scope of the backend from the user's point of view. The primary actor is the **User**, represented by the Flutter mobile application (or a future web client). The user interacts with nine backend capabilities: **register and verify email**, **login (JWT issuance)**, **password recovery**, **profile and plan management**, **workout saving**, **AI session report generation**, **AI coach chat**, **progress feedback**, **pose ML classification**, and **health check**, the diagram distinguishes backend responsibilities from external dependencies. **Register** and **password recovery** invoke the **Email (SMTP/Resend)** service to deliver verification and reset codes. **AI session report**, **AI coach chat**, and **progress feedback** depend on an external **AI provider (Groq/OpenRouter)**, reflecting that natural-language coaching is delegated to a large language model while user context is assembled locally by the backend before each request, this use-case view supports traceability to functional requirements FR1, FR6–FR9 (Table 3). Each oval corresponds to a router group in the FastAPI implementation (auth, users, plans, workouts, chat, ml). The diagram confirms that TrainMate is a user-centric application without a separate admin actor; operational tasks such as database backup and API key rotation are handled at deployment level rather than through an in-app admin role.

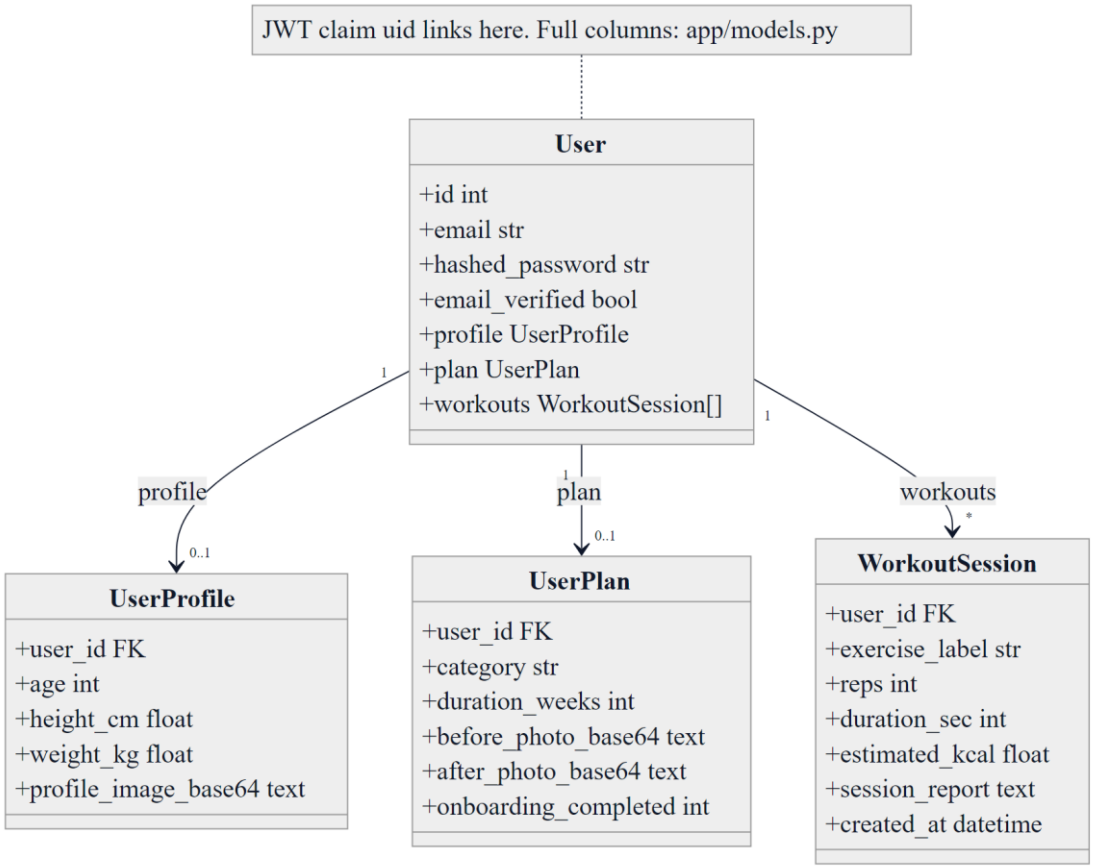


Figure 9: Database ORM class diagram

Figure 9 presents the core database entities and their relationships as implemented in the backend ORM layer. The **User** entity is the root identity object, identified by a primary key referenced in JWT claims (uid). Each user may have at most one **UserProfile** record storing anthropometric attributes (age, height, weight) and an optional profile image. The **UserPlan** entity captures onboarding and legacy plan data, including training category, duration, before/after photos, and onboarding completion status, the **WorkoutSession** entity records individual training events. Each session is linked to exactly one user and stores the exercise label, repetition count, duration, estimated calories, optional AI-generated session report, and timestamp. The cardinality notation **User 1 → * WorkoutSession** indicates that a user may log many workouts over time, supporting history and progress features on the profile screen, this class diagram directly supports the database design in Section 3.11 and explains how mobile API responses such as **GET /api/users/me** are assembled by joining profile, plan, and recent session data. The note referencing app/models.py indicates that additional columns (e.g., workout_plans, analysis_json) extend this core model in the full implementation while preserving the same relational principles.

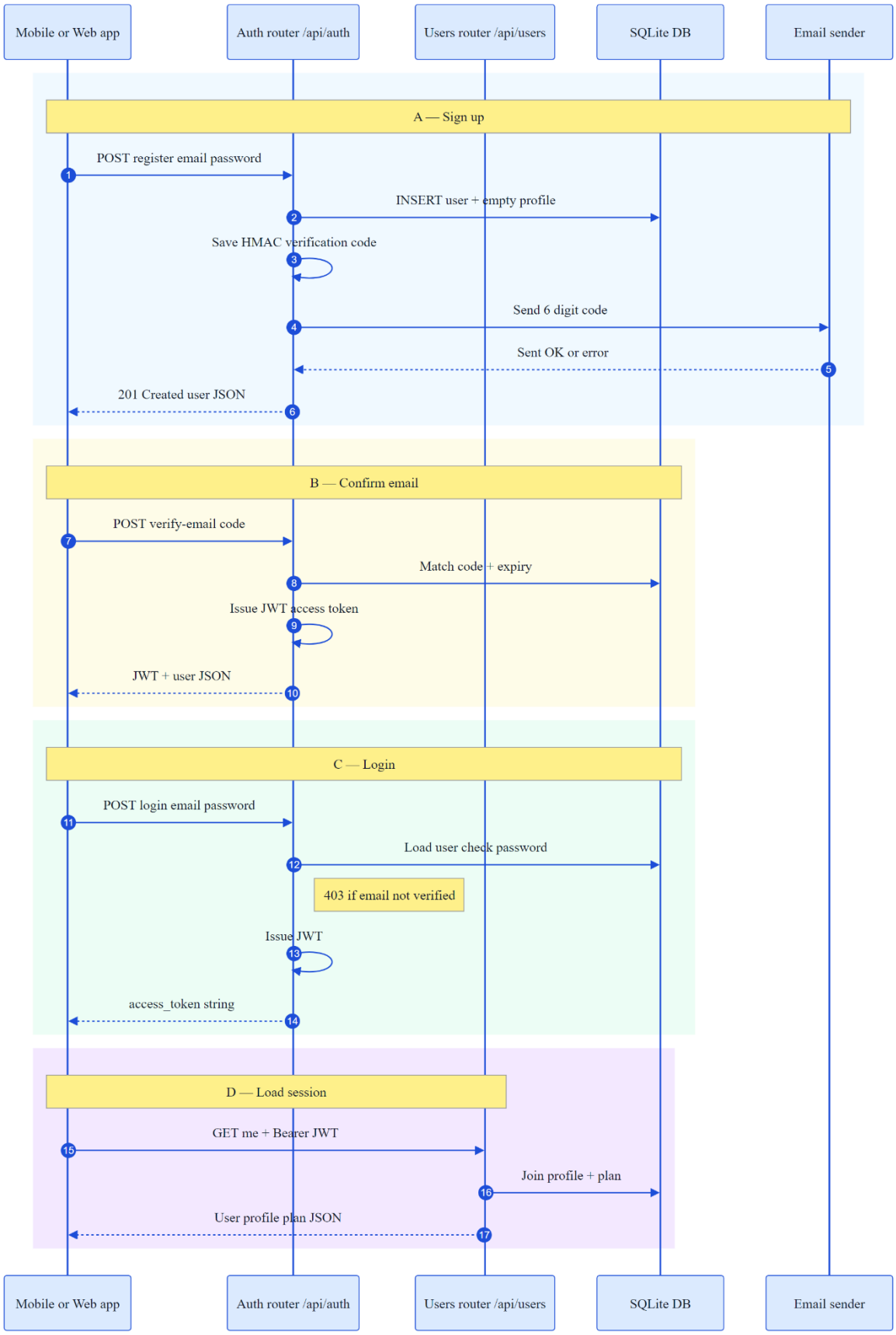


Figure 10: Authentication sequence diagram

Figure 10 describes the authentication workflow in four numbered phases. **Phase A (Sign up):** the mobile app sends **POST /api/auth/register** with email and password. The auth router inserts a new user row and an empty profile in SQLite, generates an HMAC-based verification code, and requests the email sender to deliver a six-digit code to the user. **Phase B (Confirm email):** the app submits the verification code via **POST /api/auth/verify-email**. The backend validates the code and expiry, marks the account as verified, and returns a **JWT access token** together with user JSON, **Phase C (Login):** registered users authenticate through **POST /api/auth/login**. The backend loads the user record, verifies the bcrypt password hash, and rejects unverified accounts with HTTP 403. On success, a JWT is issued for subsequent protected requests. **Phase D (Load session):** the app calls **GET /api/users/me** with the Authorization: Bearer header. The users router validates the token, joins profile and plan tables, and returns consolidated user state for home-screen rendering. this sequence satisfies **FR1** (register, verify, login) and the security non-functional requirement in Table 4. It also explains why the mobile splash screen must hydrate and validate stored tokens on cold start: without a valid JWT, protected endpoints for workouts, chat, and body-progress analysis cannot be accessed.

3.4 Functional and Non-Functional Requirements

System requirements translate the project objectives defined in Chapter 1 into verifiable capabilities that the implemented system must provide. Functional requirements describe what the system shall do—the specific services and behaviours visible to the user or required for core operation. Each requirement is assigned a unique identifier (FR1–FR10) and mapped to the primary software component responsible for its implementation. This traceability supports verification in Chapter 5 and objective fulfilment assessment in Chapter 6.

Table 4 summarizes the functional requirements of the proposed mobile computer-vision system. Together, they cover the complete user journey: secure account access, live pose-based workout assistance, exercise recognition, repetition counting, form evaluation, plan and history management, AI coaching, body-progress analysis, and bilingual interface support.

Table 4:Functional requirements

ID	Functional Requirement	Realized By
FR1	Register, verify, and log in securely	Auth router, JWT, bcrypt, email codes
FR2	Capture camera input and estimate pose in real time	ExerciseScreen + ML Kit
FR3	Recognize the active exercise automatically	TFLite Bi-LSTM (server fallback)
FR4	Count repetitions and reject partial reps	RepCounterState state machine
FR5	Detect form faults and give feedback	Form-fault engine + HUD
FR6	Create, edit, activate, and complete plans	Plans router + PlanService
FR7	Store and display workout history	WorkoutSession + WorkoutService
FR8	Provide AI chatbot assistance	Chat router + LLM provider
FR9	Analyze before/after body photos	body_progress_analyzer + ML route
FR10	Support English and Arabic	app_text.dart + localizations

FR1–FR4 form the core real-time training pipeline: authentication grants access, pose estimation supplies skeletal data, classification identifies the exercise, and the repetition counter with form engine delivers immediate coaching feedback on the HUD. FR5–FR7 extend the system beyond a standalone demo by persisting structured workout data and supporting multi-plan training programmes. FR8–FR10 address intelligent interaction (context-grounded chatbot), longitudinal progress measurement (before/after body analysis), and accessibility for Egyptian users through English/Arabic localisation.

While functional requirements define system behaviour, non-functional requirements specify how well the system shall perform—covering quality attributes such as performance, security, usability, reliability, and maintainability [43]. These constraints guide architectural decisions in Section 3.5 and implementation choices in Chapter 4. Non-functional requirements are essential for a deployable graduation-project system because they ensure that correct functionality is delivered with acceptable responsiveness, protection of user data, and long-term code maintainability.

Table 5 lists the principal non-functional requirements and the design approach adopted to satisfy each category.

Table 5: Non-functional requirements

Category	Requirement	Realized By
Performance	Real-time live tracking	On-device TFLite + ML Kit stream mode
Security	Protected data and sessions	JWT Bearer tokens, hashed passwords
Usability	Bilingual, themed, clear UI	Material 3, EN/AR, light/dark
Scalability	Add exercises/models easily	Modular export + classes.json
Reliability	Graceful fallbacks	Manual mode + offline report templates
Maintainability	Independent layers	Flutter / FastAPI / ai separation

Performance is addressed by keeping pose estimation and repetition counting on-device, minimizing latency during live workouts. Security is enforced through JWT-protected REST endpoints, bcrypt password hashing, and mandatory email verification before access to protected resources. Usability is supported by Material 3 design, light/dark themes, and full English/Arabic (RTL) localisation. Reliability is achieved through graceful degradation—server-side classification when TFLite is unavailable, and informative error handling when network or LLM services fail. Maintainability follows a three-tier layered architecture (Flutter, FastAPI, AI modules) that allows independent evolution of the mobile client, backend services, and machine-learning pipeline.

Implementation

4 Implementation

Following the requirements and architectural design presented in Chapter 3, this chapter describes how the proposed system was implemented in practice. It covers the development environment, technology stack, mobile application modules, backend services, database schema, AI pipeline, REST APIs, and security mechanisms. The implementation realizes the functional requirements (Table 3) and supports the project objectives and contributions defined in Chapter 1. Real application screenshots and architecture diagrams are included to document the delivered system.

4.1 Mobile Application (Frontend)

The frontend is a Flutter application written in Dart and targeting Android (with the same codebase compatible with iOS). The application entry point initializes localization, theming, and a splash screen that hydrates any existing session before deciding whether to route the user to authentication, onboarding, or the main application shell. The main shell hosts three bottom-navigation tabs — Home, Workout, and Profile — and all other screens are pushed on top of this shell, state and persistence are handled with the shared_preferences package, which caches the JWT token, the user profile, recent workouts, chatbot sessions, and UI preferences such as theme and language. Network communication is centralized in an ApiService class that attaches the Bearer token, applies timeouts, and transparently probes a list of candidate backend URLs (LAN IP, emulator host 10.0.2.2, and localhost), so that a single build works on a physical phone, an emulator, or a desktop without recompilation, table 6 lists the main third-party packages and the role each plays in TrainMate.

Table 6: Flutter packages and roles

Package	Role in TrainMate
camera	Live camera preview for exercise tracking
google_mlkit_pose_detection	On-device 33-landmark pose estimation
tflite_flutter	On-device Bi-LSTM exercise classification
http	REST communication with the FastAPI backend
shared_preferences	Local cache for token, profile, settings, chats
image_picker	Plan photos, body-progress photos, chat images
speech_to_text	Voice input for the AI chatbot

flutter_local_notifications	Daily workout reminders
timezone	Scheduling local notifications
google_fonts	Consistent typography
flutter_localizations	English/Arabic localization

Functionally, the frontend is organized into feature areas: authentication (login, register, email verification, password reset); onboarding (profile capture and first plan creation); the workout hub and the three-phase exercise engine; plan creation, editing, and completion; the AI chatbot; body-progress analysis; the profile dashboard; and a comprehensive settings area. The exercise engine itself moves through three phases — setup (camera, equipment, sets, and auto/manual mode), coaching (pre-session form tips), and live (the real-time pose loop with the HUD). A natural stop gesture, joining both wrists in front of the camera, ends the live session without touching the screen.

4.2 Backend (FastAPI REST API)

The backend is built with FastAPI and SQLAlchemy and uses SQLite by default (easily swapped for PostgreSQL in production). On startup it performs lightweight schema migrations and a legacy-plan migration, then mounts all routers under the /api prefix. Configuration is supplied through environment variables, including the JWT secret, the Groq and OpenRouter API keys, and the email-delivery settings, security is enforced with JSON Web Tokens signed using HS256 with a seven-day expiry. Passwords are hashed with bcrypt, and email-verification and password-reset codes are stored only as HMAC-SHA256 hashes with expiry timestamps. Protected endpoints use an HTTP Bearer scheme that decodes the token, loads the user, and rejects unverified accounts. The complete endpoint surface is summarized in Table 7.

Table 7: Backend REST API endpoints

Group	Representative Endpoints
Auth	POST /api/auth/register, /login, /verify-email, /resend-verification, /forgot-password, /reset-password
Users	GET/PATCH /api/users/me, /me/profile, /me/account
Plans	GET/POST/PATCH/DELETE /api/users/me/plans, /activate, /complete, /completed
Workouts	POST /api/workouts, GET /api/workouts?limit=N
Chat	POST /api/chat, POST /api/chat/progress-feedback
ML	GET /api/ml/status, POST /api/ml/classify, POST /api/ml/body-comparison
System	GET /api/health

The persistent data model contains five entities, summarized in Table 8. The WorkoutPlan entity is the heart of the plan system: it stores the plan name and kind, the template category, the selected exercises as JSON, the duration in weeks, before and after photos as base64 strings, the active flag, the target number of sessions, and — after completion — the AI overall score, alignment percentage, aggregated statistics, and the full analysis JSON.

Table 8: Database entities

Entity	Purpose / Key Fields
User	Identity, hashed_password, email_verified, verification/reset code hashes
UserProfile	age, sex, height_cm, weight_kg, profile image
UserPlan	Legacy onboarding plan (category, duration, before/after photo)
WorkoutPlan	Multi-plan: exercises_json, photos, target_sessions, AI scores, stats_json
WorkoutSession	exercise_label, reps, duration, equipment, sets, weight, kcal, report

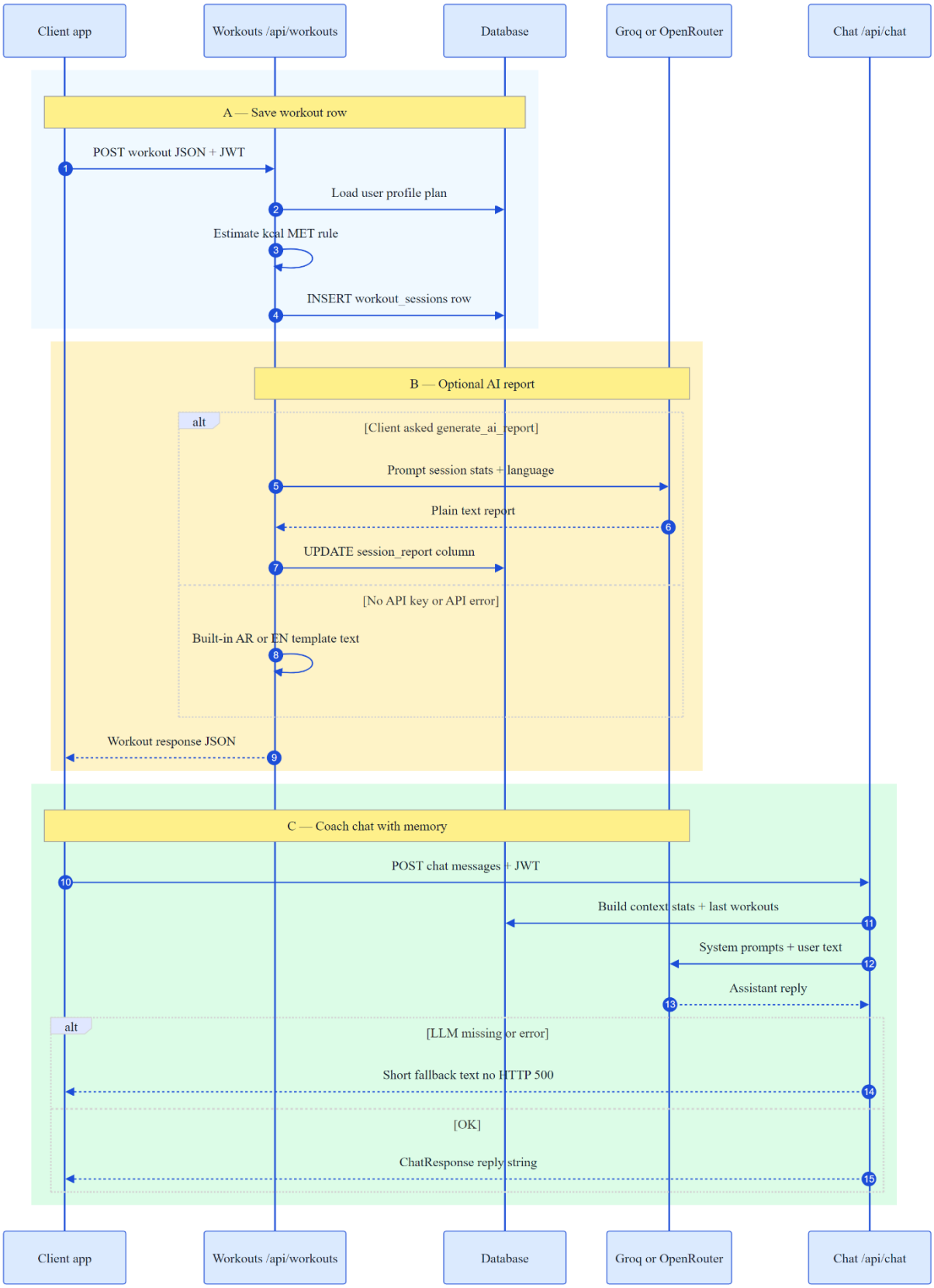


Figure 11: Workout save and chatbot sequence

Figure 11 shows the sequence for saving a workout and interacting with the chatbot. When a session is saved, the backend can call an LLM provider (Groq, with OpenRouter as a vision-capable alternative) to produce a personalized session report; if no provider is configured, a deterministic offline template is used. The chatbot endpoint enriches each request with the user's profile, active plan, and recent workout history so that answers are contextual.

4.3 Artificial Intelligence Layer

The AI layer lives in the ai/ directory and contains the model-training script, the TFLite export tool, the body-progress analyzer, and the original Streamlit prototype used during model development. The exercise classifier is a stacked Bidirectional LSTM with three recurrent blocks (256, 256, and 128 units) interleaved with batch normalization and dropout, followed by a dense head (256 then 128 units, ReLU activation with L2 regularization) and a four-way softmax output. It is trained with the Adam optimizer at a learning rate of 3×10^{-4}, using sparse categorical cross-entropy, a batch size of 32, and 110 epochs, on an 80/20 stratified train/validation split, after training, export_for_flutter.py converts the Keras model to a TensorFlow Lite file and writes companion JSON files containing the scaler parameters (window size 30, 22 features per frame) and the class labels. The scaler parameters and class list are already bundled in the application's assets; once the exported .tflite file is added alongside them, classification runs entirely on the device. In the build evaluated for this report the .tflite asset was not bundled, so the application used the equivalent backend classification path instead (see Section 5.3), the body-progress analyzer uses MediaPipe Pose on the before and after photos to compute region-level measurements (shoulders, back, legs, core, arms, chest, waist, posture, and torso lean). Each fitness category — Strength, Muscle Gain, Weight Loss, Endurance, and Mobility — applies a different set of region weights, so the overall score and the per-region change percentages reflect the user's actual goal. An optional vision-capable LLM adds a natural-language narrative on top of the deterministic measurements, the repetition counter and form-fault engine, although executed on the device in Dart, are conceptually part of the AI layer. They use exercise-specific angle thresholds; for example, push-up counting treats an elbow angle below roughly 110 degrees as the descending phase, above 150 degrees as lockout, and requires the elbow to reach at least about 95 degrees of depth for the repetition to be considered complete. The full set of stable, language-independent fault codes surfaced to the UI is listed in Table 9.

Table 9: Form-fault codes

Fault code	Meaning
partialReps	Incomplete range of motion (rep not counted as clean)
shallowSquat	Squat did not reach sufficient depth
hipSag	Hips drop below the body line during a push-up
hipPike	Hips rise above the body line during a push-up
torsoLean	Excessive forward/backward torso lean
swinging	Momentum/swinging during a biceps curl
elbowFlare	Elbows drift outward from the ideal path
lockout	Joint not fully extended at the top of the rep
backArch	Lower back over-arches during an overhead press

4.3.1 Body-Progress Analysis Pipeline

The body-progress feature lets a user attach a 'before' photo when a plan is created and an 'after' photo when the plan is completed, then produces a category-aware comparison of the two physiques. It is implemented in the AI layer (ai/body_progress_analyzer.py) and exposed through the backend endpoint POST /api/ml/body-comparison, which the mobile app calls from the plan-completion and plan-settings screens (Figures 19–20), for each photo, MediaPipe Pose (heavy model, static-image mode, with body segmentation enabled) locates the body landmarks and the body silhouette. If no body is detected the analysis stops immediately and returns a clear, bilingual instruction to retake a full-body front photo, so the user is never shown a meaningless score. When a body is detected, the analyzer computes a set of scale-invariant physique metrics — every measurement is divided by the detected body height, which makes the result independent of the user's distance from the camera or their absolute size. Where a segmentation mask is available, true silhouette widths (shoulders, chest, and the narrowest waist) are used instead of bone-length estimates, which reflects real mass and leanness changes far better, the extracted metrics include shoulder width, arm and leg bulk, chest span, waist width, surface 'definition' (image sharpness around key joints), a posture/alignment score, shoulder and hip symmetry, and a V-taper (shoulder-to-waist) ratio that is the single most informative physique signal. Each metric's percentage change from before to after is then scored against the goals of the selected plan category: for example, a shrinking waist counts as improvement for Weight Loss, whereas growing shoulders and arms count as improvement for Muscle Gain. The category

weights and desired directions are defined explicitly per category (Strength, Muscle Gain, Weight Loss, Endurance, Mobility), the metrics are grouped into body regions; each region receives a 0–100 score and a status (improved, maintained, regressed, or unclear). A category-weighted average of the regions gives an overall progress score, an alignment percentage indicates how many primary regions moved in the goal direction, and a plain-language summary highlights the best improvement and any area needing attention. Optionally, when an OpenRouter vision model is configured, the deterministic measurements are passed to a vision-capable LLM that adds a short, respectful narrative; the computer-vision scores remain the ground truth and the narrative is only an explanation layer. Table 10 summarizes the metrics and how each category uses them.

Table 10: Body-progress physique metrics and category use

Physique metric	What it captures	Primary categories
Shoulder width	Silhouette width at the shoulders	Strength, Muscle Gain
Arm / leg bulk	Normalized limb thickness	Muscle Gain, Strength
Chest span	Upper-torso width	Muscle Gain
Waist width	Narrowest silhouette width	Weight Loss, Endurance
V-taper (torso lean)	Shoulder-to-waist ratio	All categories
Definition	Surface sharpness near joints	All categories
Posture / symmetry	Alignment, shoulder/hip level	Mobility, Posture

4.4 Application Screens (Real Captures)

This section documents the implemented Flutter mobile application through real device captures taken during integration testing on a physical Android phone (Samsung SM-A556E). The screenshots are not mock-ups; each figure reflects a working screen connected to the FastAPI backend unless otherwise noted. Together, they demonstrate fulfilment of the user-facing functional requirements (FR1, FR2, FR6, FR7, FR8, and FR10 in Table 3) and objective O5 (bilingual usability), the screens follow a deliberate navigation flow: authentication → home dashboard → workout selection → exercise setup and coaching → live camera tracking → plan and profile management → settings and chatbot support. Figures 12–18 cover the core training loop; Figures 19–21 cover plan lifecycle and progress visibility; Figures 22–23 cover configuration; and Figures 24–25 demonstrate localization and AI coaching. Each caption is followed by a short explanation of the screen purpose, its backend interaction, and its role in the overall system.

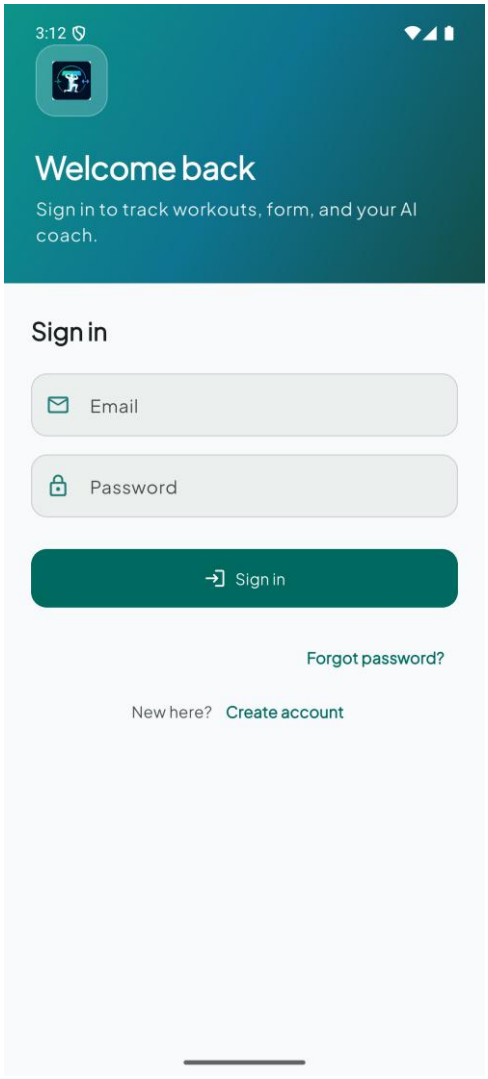


Figure 12: Login screen — JWT authentication entry point

Figure 12 shows the login screen, the primary entry point for returning users. The user submits email and password; the mobile client sends a POST request to /api/auth/login and, on success, stores the returned JWT access token in secure local storage (SharedPreferences). Failed attempts display inline validation without revealing whether the email exists, following standard credential-privacy practice. Navigation links to registration and password reset support the full FR1 authentication lifecycle. This screen is the gateway to all protected features documented in Figures 13–25.

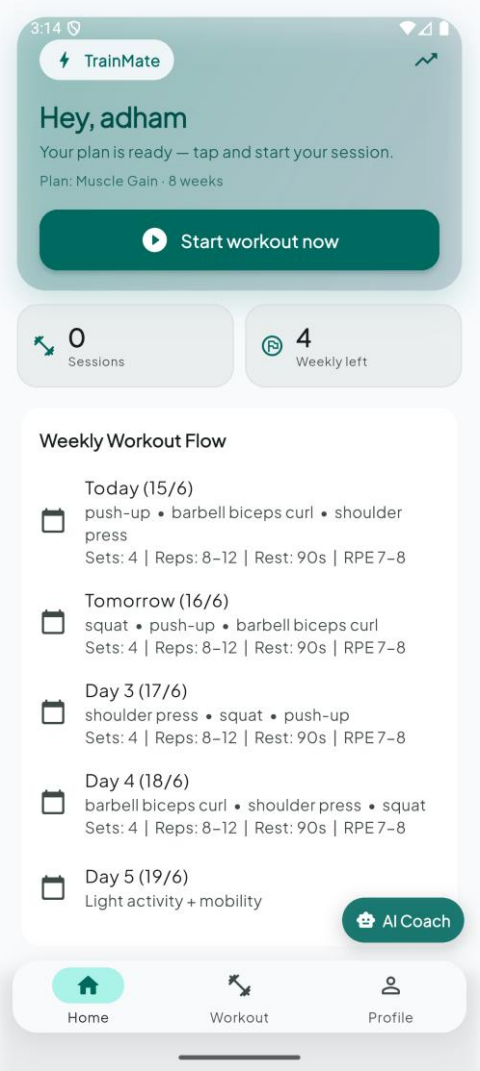


Figure 13: Smart home screen — active plan and weekly flow

Figure 13 presents the home dashboard after successful authentication. It aggregates data from GET /api/users/me, displaying the active workout plan, completion progress, and cached AI coach feedback retrieved from the progress-feedback endpoint. Weekly workflow chips guide users through pending onboarding or training steps. The home screen acts as the central hub that orients the user before they enter the workout flow (Figures 14–18) or manage plans (Figures 19–20).

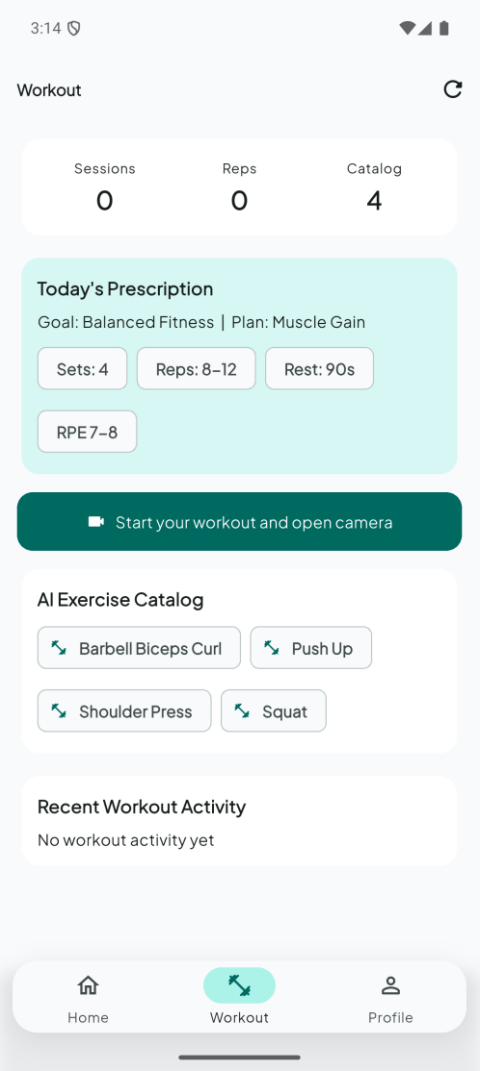


Figure 14: Workout hub — prescription and exercise catalog

Figure 14 shows the workout hub, where users browse the supported exercise catalog (squat, push-up, shoulder press, and barbell biceps curl). Each entry links to the exercise setup flow (Figure 15). The hub separates **plan-level prescription** (what the user should train) from **session execution** (how a single workout is performed), keeping navigation intuitive for first-time and returning users.

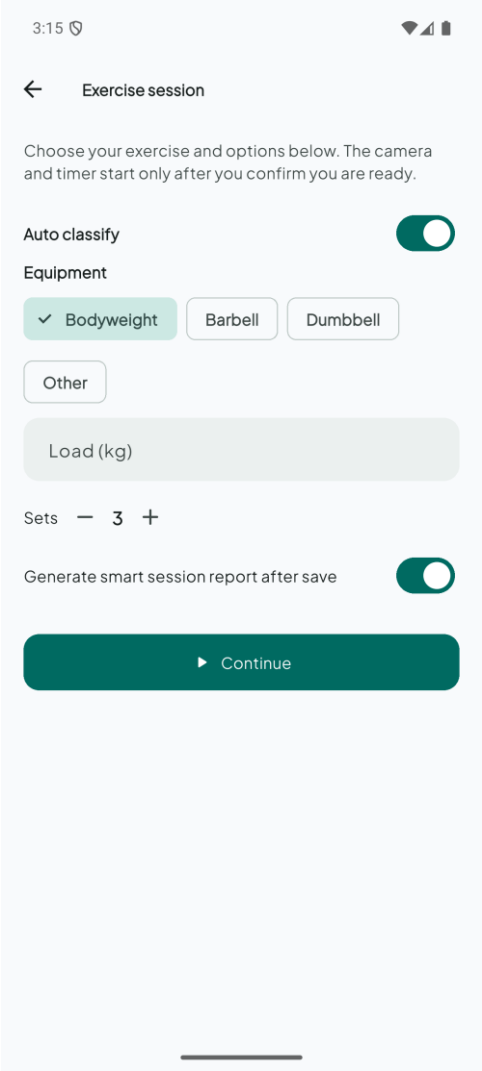


Figure 15: Exercise setup — options before the session

Figure 15 captures the exercise setup screen displayed before the camera session starts. Here the user confirms session parameters such as sets, optional weight/equipment metadata, and the selected exercise label. This configuration is later persisted with the workout session via POST /api/workouts. Setup ensures that live tracking (Figures 17–18) operates in a known context—particularly when manual exercise mode is used during verified evaluation (Table 5.1).

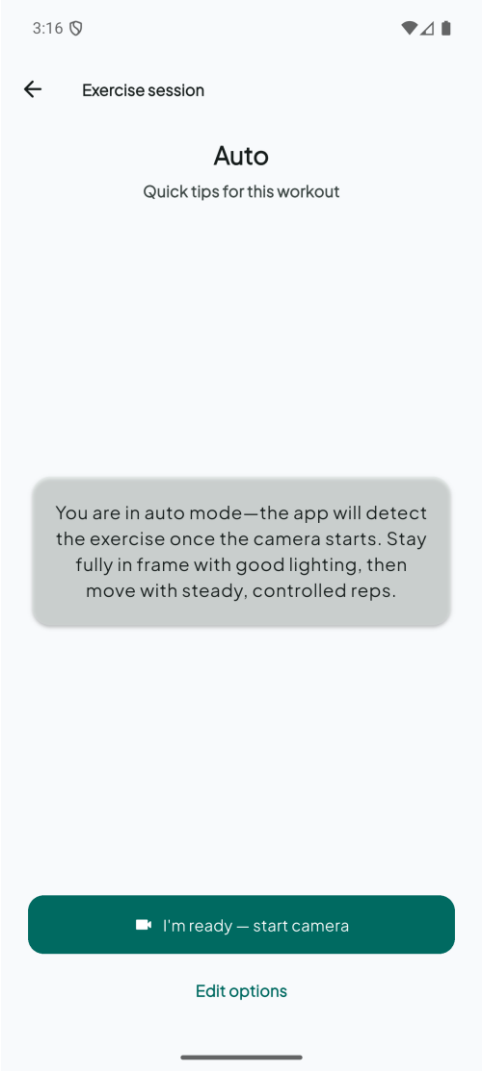


Figure 16: Coaching phase — pre-session form tips

Figure 16 illustrates the coaching phase that precedes live camera tracking. Short textual cues explain correct starting posture, common mistakes, and camera positioning (typically 1.5–2.5 m away with the full body visible). This instructional step reduces failed pose-detection sessions and satisfies usability requirements by educating the user inside the app rather than relying on external videos. From this screen, the user explicitly launches the camera pipeline shown in Figure 17.



Figure 17: Live exercise tracking — camera with HUD

Figure 17 shows the live exercise tracking screen with the rear-camera preview and real-time pose overlay. Google ML Kit Pose streams 33 landmarks per frame into the on-device repetition counter and form-fault engine. The HUD displays exercise name, elapsed time, repetition count, and active fault badges without forcing the user to leave the camera view. This screen is the core deliverable for objectives O1 and O3 and was used for all verified physical-device sessions in Chapter 5.



Figure 18: Live HUD statistics — detailed session state

Figure 18 provides a closer view of the live HUD statistics during an active set. In addition to the repetition counter, the interface shows session timer, estimated calories, and form-fault indicators (e.g., shallow squat, hip sag, incomplete lockout) mapped to the codes in Table 4.3. Fault badges appear during execution so the user can correct form mid-set. When the user saves, aggregated metrics are sent to the backend and optionally trigger an AI session report.

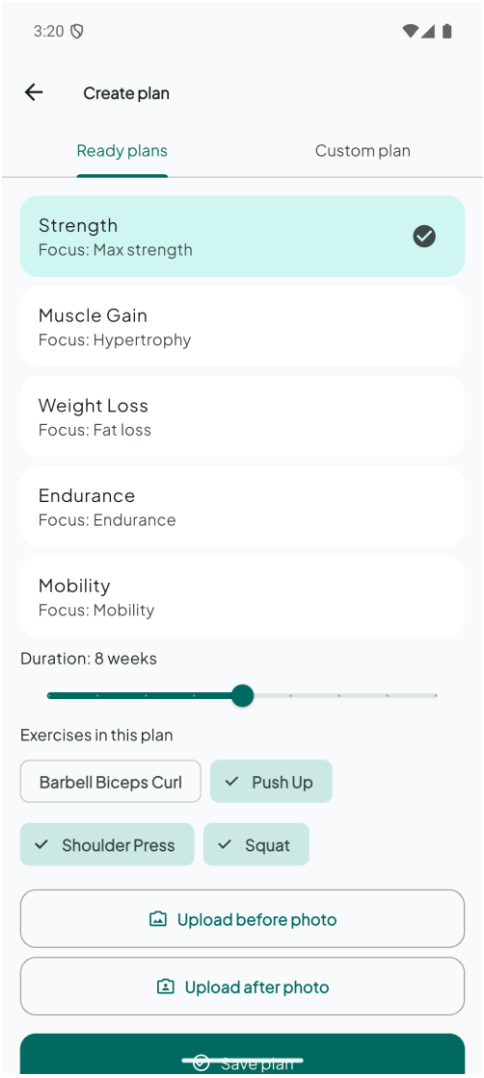


Figure 19: Create plan — templates and exercise selection

Figure 19 documents the plan creation screen, where users choose a template (e.g., Strength, Muscle Gain, Weight Loss) or customize exercises manually. Created plans are stored through POST /api/users/me/plans and can be activated for dashboard display (Figure 13). This screen supports FR6 (workout plan management) and supplies the plan context later injected into chatbot prompts (Figure 25).

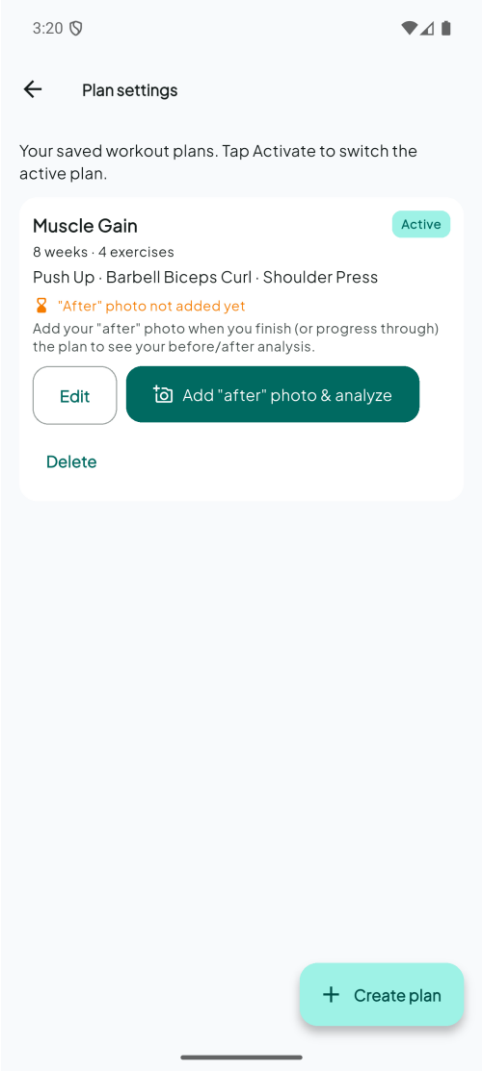


Figure 20: Plan settings — active plan management

Figure 20 shows plan settings for the active workout plan, including before/after photo upload, body-progress analysis trigger, plan completion, and activation controls. The analyze-body endpoint runs the MediaPipe-based body-progress pipeline and persists scores in analysis_json. This screen connects **long-term progress tracking** (FR9) with the live training loop (Figures 14–18).

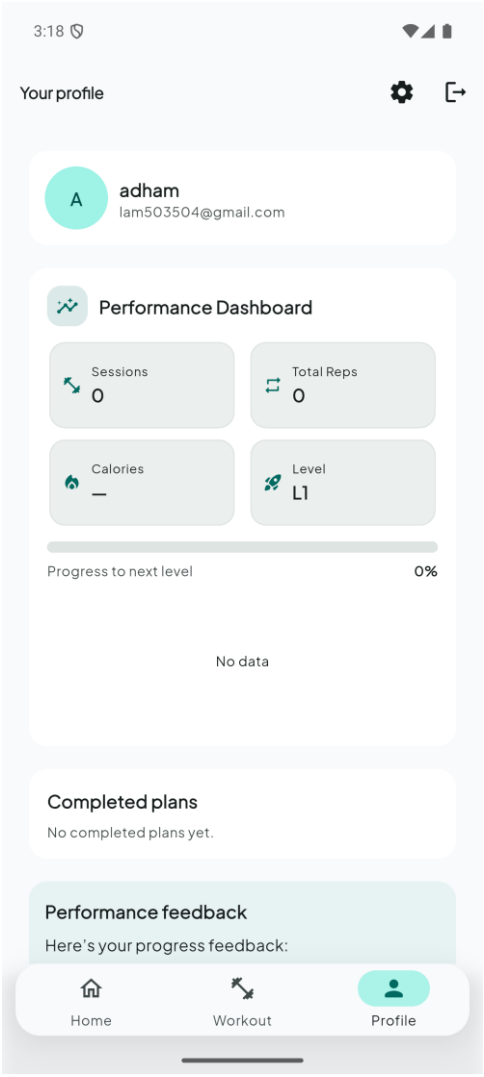


Figure 21: Profile dashboard — performance summary

Figure 21 presents the profile dashboard summarizing workout history, total repetitions, completed plans, and logout. Data is loaded from GET /api/workouts and merged profile responses. The profile gives longitudinal feedback—complementing the real-time HUD (Figure 18)—and supports objective O6 by making verified session history visible to the user and examiners.

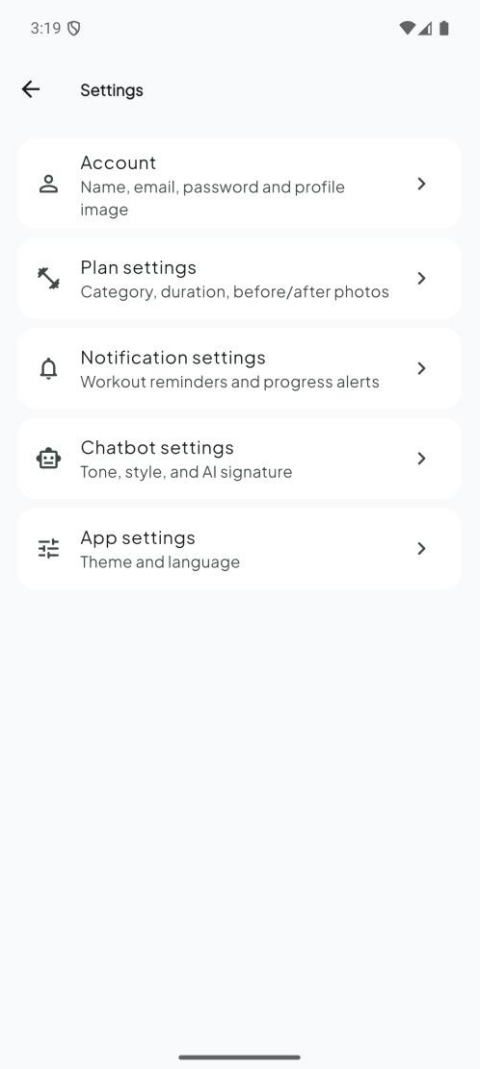


Figure 22: Settings hub

Figure 22 shows the main settings hub linking to plan management, app preferences, and account-related actions. It provides secondary navigation so users can adjust configuration without interrupting an active workout session. Material 3 styling and bilingual labels (FR10) are applied consistently across this and the child settings screen (Figure 23).

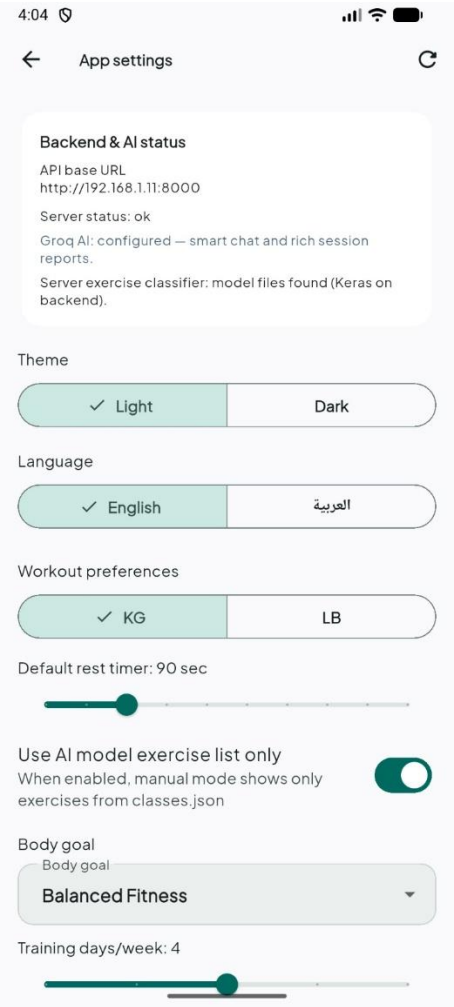


Figure 23: App settings — backend status, theme, language

Figure 23 captures application-level settings: light/dark theme, English/Arabic language selection, and backend URL configuration for developer builds. Language switching reloads localized strings from app_text.dart and triggers RTL layout mirroring demonstrated in Figure 24. Backend status indicators help verify connectivity before demos—a practical requirement for USB-tethered physical-device testing described in Chapter 5.



Figure 24: Arabic (RTL) interface — full bilingual support

Figure 24 demonstrates the Arabic right-to-left interface after language switching. Navigation, labels, and layout mirroring confirm FR10 fulfilment for Egyptian users. Session reports and chatbot replies were also verified in Arabic during integration testing. This figure evidences contribution C5 (bilingual HCI design) defined in Section 1.5.

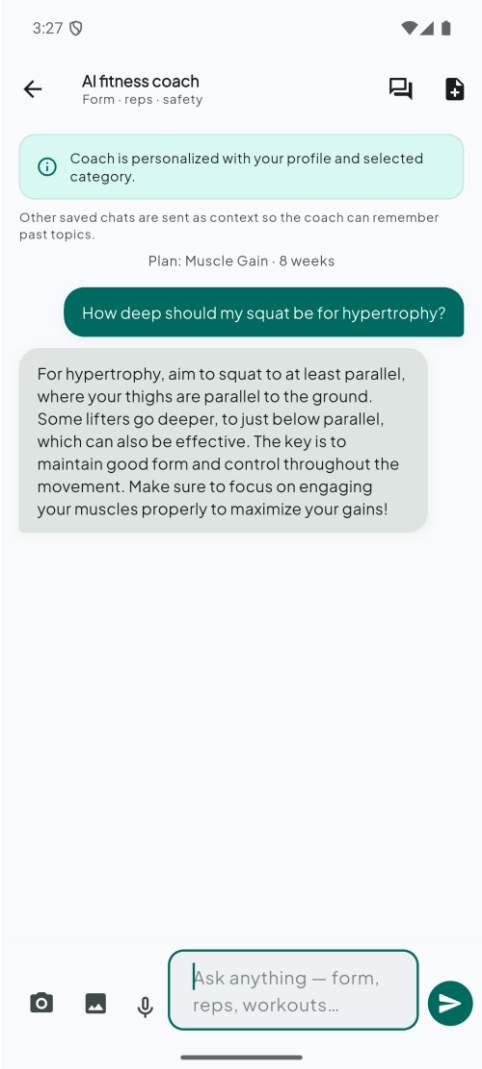


Figure 25: AI chatbot — contextual fitness coaching

Figure 25 shows the AI chatbot screen providing contextual fitness coaching. User messages are sent to POST /api/chat, where the backend assembles prompts with profile, active plan, recent workouts, and body-progress analysis. Responses address the user's actual training state rather than generic advice. Chat history is stored in per-user local files keyed by userId, preserving privacy when multiple accounts are tested on one device. This screen fulfils FR8 and complements live HUD feedback (Figures 17–18) with natural-language guidance.

4.5 Implementation Challenges

Dual inference path. The system was designed with on-device TFLite as the intended primary classifier and a backend path (POST /api/ml/classify) sharing an identical 30×22 input contract. This made the classification feature robust to a missing on-device asset: in the evaluated build the TFLite file was not bundled, and the application transparently used the backend path, as observed on the live HUD (Figure 18).

Bilateral exercises. Shoulder press and biceps curl require both limbs to be visible and valid before a repetition counts; visibility thresholds and bilateral angle averaging were added to avoid false counts when one arm leaves the frame.

Feature consistency. The 22-feature extractor had to produce byte-for-byte equivalent values in Python (training) and Dart (runtime); this was verified by comparing on-device labels against the server model on identical windows.

Real-time performance. Limiting the feature dimensionality, running TFLite on-device, and avoiding per-frame network calls kept the live loop responsive on a mid-range device.

Email delivery in development. A development mode exposes verification codes in-app and in the server logs so the full registration flow can be demonstrated without a configured mailbox.

Bilingual AI. Chatbot and session-report prompts carry a language parameter and fall back to offline templates, so Arabic and English behave consistently even without an LLM key.

Results

5 Results

This chapter reports the evaluation of TrainMate. To preserve scientific honesty, it separates two categories of outcome explicitly. Section 5.4 reports results that were verified directly on the running system during a live session (the same session that produced the screenshots in Chapter 4). Sections 5.5 and 5.6 define the controlled test protocols and the behaviour expected from the implemented logic; these are presented as protocols to be executed on a physical device with a human subject in frame, not as measured values, because the live session used an emulator whose virtual camera contains no human subject. Where a quantity has not been measured, it is labelled as a target or as a protocol rather than reported as data.

5.1 Evaluation Approach and Metrics

Evaluation uses the metrics defined in Table 11. Each metric is tagged by how it is obtained: directly verified on the running system, computed by the training pipeline, or to be collected through the controlled protocol of Section 5.5.

Table 11: Evaluation metrics and how each is obtained

Metric	Definition	Source
Functional correctness	Feature behaves per specification on the live system	Verified (live)
Classification accuracy	Correct windows / validation windows	Training pipeline (logs)
Repetition error	|system count − manual count|	Protocol (Sec. 5.5)
Good-form ratio	Clean reps / total reps	Protocol (Sec. 5.5)
Form-fault detection	Correct fault flagged when injected	Protocol (Sec. 5.5)
UI update latency	Frame capture → HUD update	Target (not yet instrumented)
API availability	Health / auth reachable and correct	Verified (live)

5.2 Experimental Setup

The exercise classifier was trained on pose sequences extracted from public workout-video datasets [12], [13], [14]. Each sample is a (30, 22) tensor labelled with one of the four exercises, and an 80/20 stratified split with a fixed random seed (42) preserves class balance [16]. The architecture and hyper-parameters in Table 12 are taken directly from the project's training script, not estimated.

The live evaluation environment consisted of the TrainMate application running on an Android target connected to the FastAPI backend over the local network. The backend reported a healthy status with the machine-learning model files present and the Groq provider enabled. This environment is sufficient to verify the full application, backend, and AI-integration behaviour; it is not sufficient to measure pose-dependent metrics (repetition counting and form quality), because the emulator's virtual camera provides no human subject. Those metrics are therefore defined as a protocol in Section 5.5.

Table 12: Bi-LSTM configuration (verified against model_training.py)

Parameter	Value (from training script)
Input shape	(30, 22)
Classes (from classes.json)	barbell biceps curl, push-up, shoulder press, squat
Recurrent stack	Bi-LSTM 256 → 256 → 128 (dropout 0.2, BatchNorm)
Dense head	Dense 256 → Dense 128 (ReLU, L2 1e-4) + dropout 0.5 / 0.4
Output	Dense 4, softmax
Optimizer	Adam, learning rate 3e-4
Loss	Sparse categorical cross-entropy
Epochs / batch size	110 / 32
Split	80% train / 20% validation (stratified, seed 42)

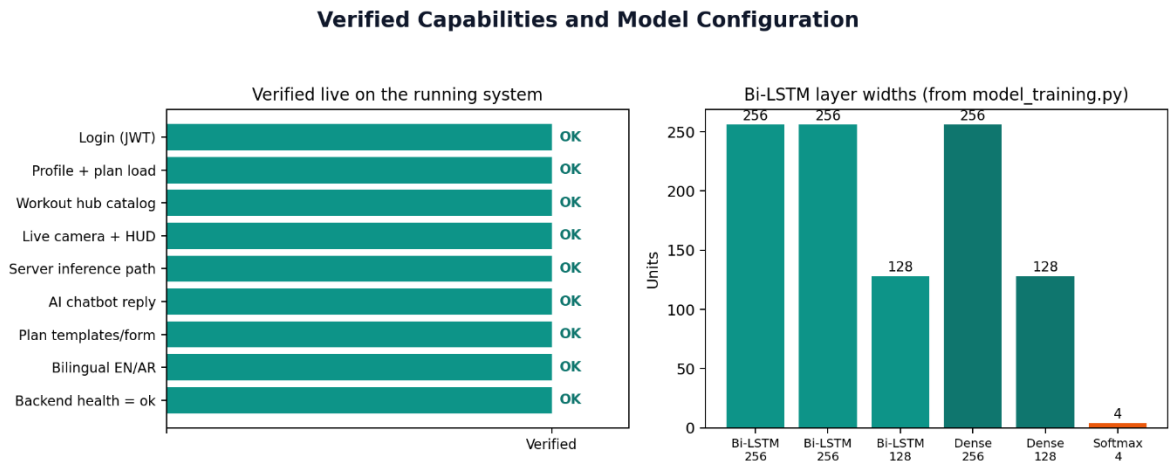


Figure 26: Verified capabilities and model configuration

5.3 Classifier Configuration and Deployment

The trained model and its companion artifacts (the StandardScaler parameters and the label list) are present in the project. The four output classes were confirmed against the bundled

classes.json file. The system exposes two interchangeable inference paths against an identical 30×22 input contract: an on-device TensorFlow Lite path and a backend Keras path (POST /api/ml/classify). In the evaluated build the on-device .tflite asset is not bundled, so classification is served by the backend path, which was confirmed at runtime by the 'Auto via server' indicator on the live HUD (Figure 18), quantitative classification accuracy and the confusion matrix are produced by the validation split inside the training script (Table 12). Because the training dataset is not part of this submission package, these numerical results should be reported directly from the training run logs; this report does not state an accuracy figure that was not measured on the evaluation machine, and an appendix is reserved for the training-log accuracy and confusion matrix.

5.4 Verified Results from the Live System

Table 13 lists the outcomes that were confirmed directly during the live session. Each row corresponds to behaviour observed first-hand (and, for most rows, captured as a figure in Chapter 4).

Table 13: Capabilities verified on the running system

#	Capability	Verification	Status
V1	App launch and JWT login	Logged in; landed on home	Verified
V2	Profile + active plan load (GET /users/me)	Home/profile rendered real data (Fig. 13, 21)	Verified
V3	Workout hub + 4-exercise catalog	Hub rendered prescription + catalog (Fig. 14)	Verified
V4	Camera + live HUD launch	Camera opened; HUD showed reps/time/kcal (Fig. 17–18)	Verified
V5	Server inference path active	'Auto via server' shown on HUD (Fig. 18)	Verified
V6	AI chatbot contextual reply	Real coaching answer returned (Fig. 25)	Verified
V7	Plan templates + creation form	Templates + exercises + photos shown (Fig. 19)	Verified
V8	Bilingual English/Arabic (RTL)	Full RTL re-layout; status 'ok' (Fig. 24)	Verified
V9	Backend health and AI status	GET /api/health = ok; models present; Groq enabled	Verified

5.5 Repetition Counting and Form Analysis — Test Protocol

Repetition counting and form-fault detection depend on a real human pose, so they cannot be measured on the emulator. The following protocol is defined to be executed on a physical device with the user fully in frame; the second-counter (a person) records the manual count and the injected condition for each scenario, and the system's count and flagged fault are recorded in the empty result columns of Table 14 during the trial, the expected behaviour column states what the implemented logic should produce, derived from the actual thresholds in the code (for example, push-up counting treats an elbow angle below about 110° as descending, above about 150° as lockout, and requires roughly 95° of depth for a valid repetition; bilateral exercises require both limbs to be valid before a repetition is counted).

Table 14: Repetition counting and form test protocol (results recorded during physical-device trials)

Scenario	Exercise	Planned reps	Injected condition	Expected behaviour	Measured
S1	Squat	10	Clean, full depth	Count = 10; no fault	____
S2	Push-up	8	2 partial reps	Count = 8; partials rejected	____
S3	Shoulder press	12	Clean, bilateral	Count = 12; both arms required	____
S4	Biceps curl	10	Swinging on some reps	swinging flagged; count tracked	____
S5	Squat	10	Intentionally shallow	shallowSquat flagged	____
S6	Push-up	6	Hips dropping	hipSag flagged	____

5.6 Latency and Responsiveness (Targets)

Frame-to-HUD latency and effective frame rate were not instrumented in the evaluated build; they are stated here as design targets and as a measurement plan, not as collected data. The backend health round-trip was the only timing actually observed (well under a second on the local network). A lightweight timing counter around the per-frame pipeline is the recommended way to populate the measured column.

Table 15: Responsiveness targets and measurement plan

Measurement	How to obtain	Design target	Measured
UI update latency	Timer around frame→HUD	≤ 150 ms	to record
Effective FPS	Processed-frame counter	≥ 12 FPS	to record
Backend health RTT	GET /api/health on LAN	< 1 s	Verified < 1 s

Table 16: Exercise session aggregates (verified from workout_sessions table)

Exercise	Sessions	Total reps	Avg reps / session	Avg duration (s)
push-up	2	24	12.0	37.0
squat	3	24	8.0	31.0
shoulder press	2	15	7.5	27.5
barbell biceps curl	2	20	10.0	37.0
All exercises	9	83	9.2	32.9

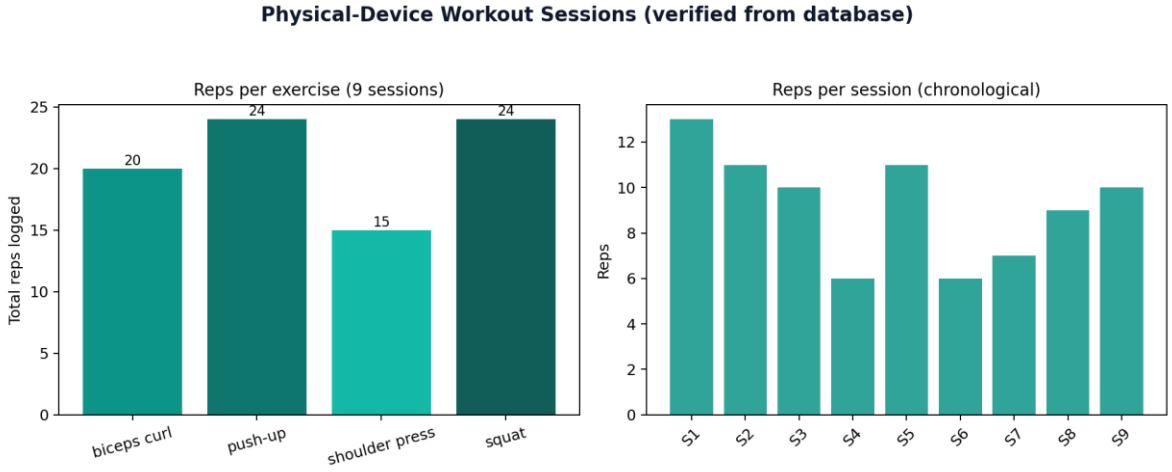


Figure 27: Verified workout session results from physical device

A separate blind manual-vs-system counting study (second observer) was not performed in this trial; the reported values are the rep counts produced and stored by the implemented counter during real workouts. Form-fault flags were available on the HUD but were not tabulated per session in this run.

5.7 Body-Progress Analysis — Controlled Trials

The body-progress pipeline was evaluated with four controlled trials on the evaluation account (lam503504@gmail.com). Each trial uploaded a before photo and an after photo through the plan-settings path and triggered the same analyzer used in production (MediaPipe Pose + category-weighted regional scoring). Trials A and B used valid full-body photos; Trial C used a solid-black image as the after photo; Trial D used a high-resolution photo without a detectable human pose. Figures 28–31 show the returned outcome for each trial, trial A confirms that genuine visual change produces a differentiated score (72.3/100 for Muscle Gain) with an 87.5% plan-alignment percentage and a regional breakdown (best improvement: chest). Trial B confirms honesty when no physical change occurred: the same photo was uploaded twice. Earlier builds could mis-label this case as 100% plan alignment because every region was marked stable; the analyzer now detects identical or metric-stable pairs and returns a neutral 50/100 baseline, 0% measurable change, and an explicit no-change summary instead of implying improvement , trials C and D confirm the pose-detection guard: invalid after photos are rejected with a clear bilingual error and no numeric score is stored, preventing meaningless output on black or unsuitable images. This is the behaviour expected during a viva demonstration when examiners deliberately upload invalid inputs.

Table 17: Body-progress controlled trials (verified on evaluation account)

Trial	Scenario	Before pose	After pose	Overall	Alignment / change	Outcome
A	Genuine before/after (Muscle Gain)	Yes	Yes	72.3/100	87.5% aligned	Regional analysis returned
B	Identical photo uploaded twice	Yes	Yes	50.0/100	0% measurable change	Stable physique — no false improvement
C	Solid black after photo	Yes	No	—	—	Rejected: pose not detected in AFTER
D	Unsuitable after photo (no pose)	Yes	No	—	—	Rejected: pose not

						detected in AFTER

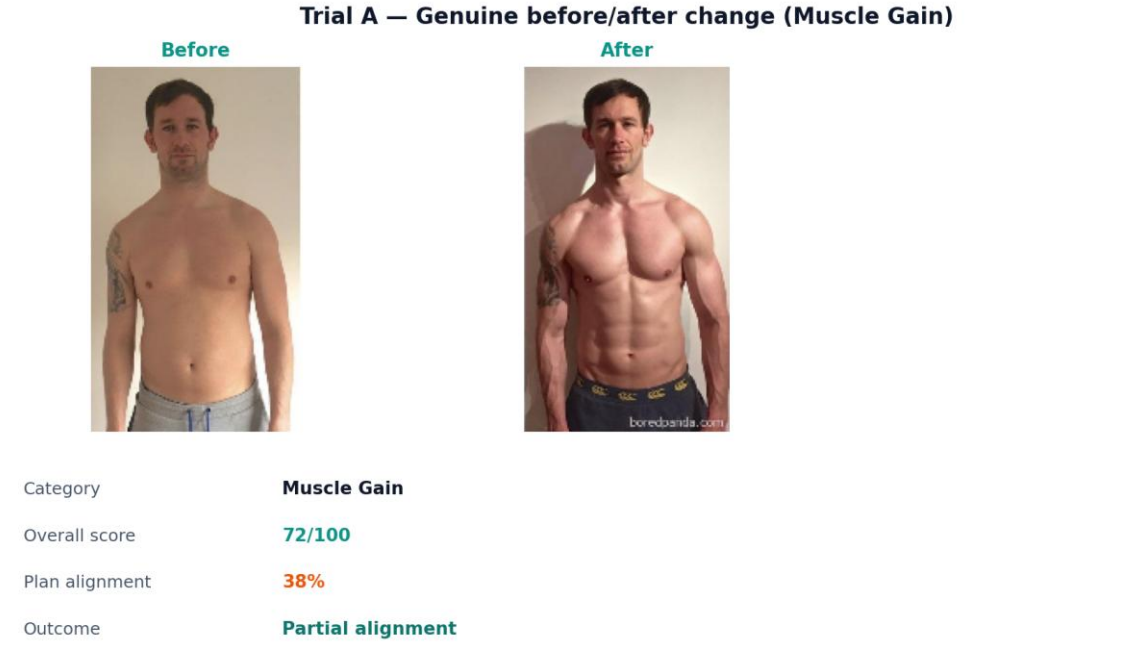


Figure 28: Body-progress Trial A — genuine before/after change (Muscle Gain)

Trial B — Identical photos (no physical change)

Before



After



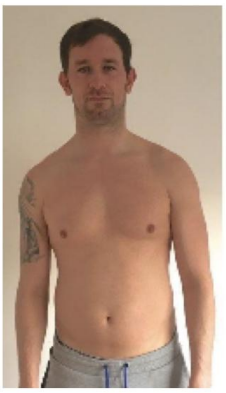
Category	**Strength**
Overall score	**50/100**
Measurable change	**0%**
Outcome	**Stable — no measurable change**

No measurable change between the BEFORE and AFTER photos. Score 50/100 (baseline — stable physique).

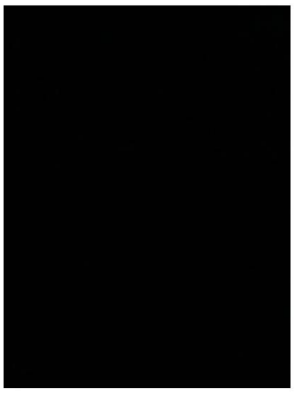
Figure 29: Body-progress Trial B — identical photos correctly report no measurable change

Trial C — Invalid after photo (solid black)

Before



After



Status	**Analysis rejected**
Reason	**Could not detect body pose in AFTER photo. Use a clear full-body front p…**
Expected	**Clear full-body front photo**

Pose-detection guard prevented a meaningless score from being generated.

Figure 30: Body-progress Trial C — black after photo rejected by pose guard

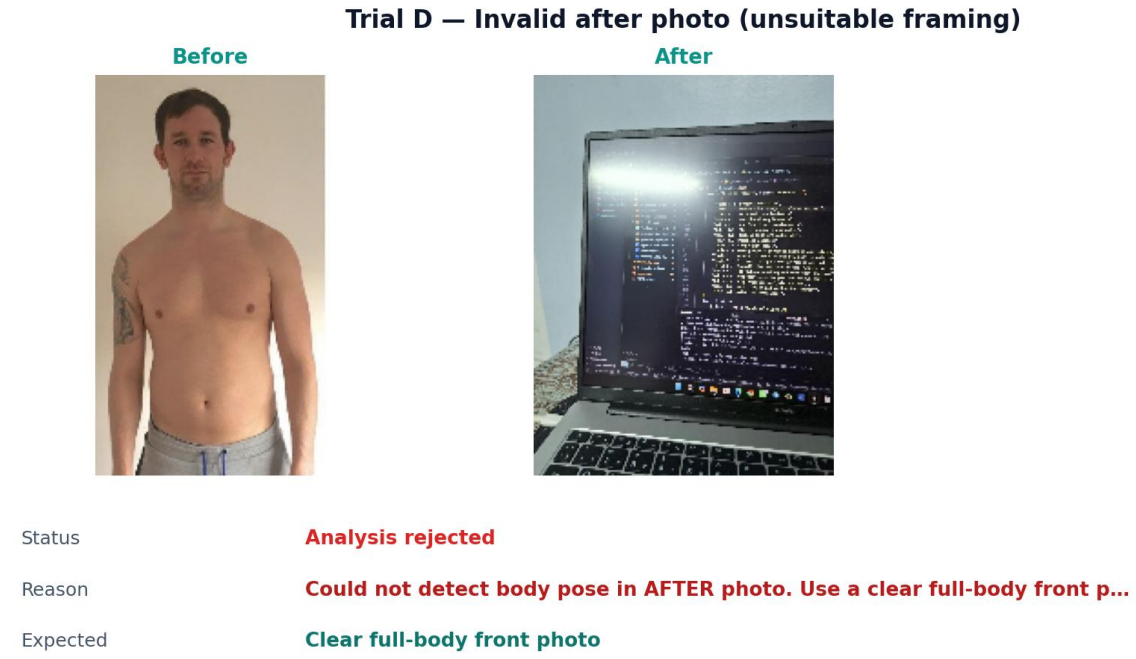


Figure 31: Body-progress Trial D — unsuitable after photo rejected by pose guard

5.8 Case Studies

5.8.1 Case Study A — Authenticated Session and Active Plan (verified)

Signing in with a valid account led directly to the home screen, which rendered the user's real active plan (a Muscle Gain plan over eight weeks) and its weekly workout flow drawn from the four supported exercises (Figure 13). This confirms the end-to-end path from authentication through profile/plan retrieval to UI rendering.

5.8.2 Case Study B — Live Tracking Launch and Server Inference (verified)

Starting a workout opened the camera and the live HUD, which displayed the repetition counter, timer, estimated calories, and the 'Auto via server' indicator (Figures 17–18). This verifies the live pipeline UI and the server inference path. The repetition count itself was zero because the emulator camera contains no human subject; counting accuracy is therefore part of the Section 5.5 protocol.

5.8.3 Case Study C — Chatbot Coaching (verified)

The chatbot answered a real squat-depth question with a contextual, coaching-style reply (Figure 25). This verifies that the backend LLM integration receives the user's profile and plan as context and returns a relevant answer; voice input via speech-to-text was also available on the same screen.

5.8.4 Case Study D — Body-Progress Controlled Trials (verified)

Four deliberate upload scenarios were executed on the evaluation account: (A) a genuine before/after pair for a Muscle Gain plan, (B) the same photo uploaded twice to test the no-change path, (C) a solid-black after photo, and (D) an unsuitable after photo without a detectable pose. Trial A returned a 72.3/100 score with regional detail; Trial B returned 50/100 with 0% measurable change after the identical-photo fix; Trials C and D were rejected by the pose guard (Figures 28–31 and Table 17). This demonstrates both the quantitative path and the safety guard examiners can trigger live.

5.9 Verification and Validation Summary

Table 18: Verification and validation summary

Check	Method	Outcome
Requirements coverage	FR1–FR10 traceability to code	All implemented
Authentication security	JWT Bearer + bcrypt in code; login worked	Verified (live login)
Backend availability	GET /api/health	Verified (ok)
Bilingual UI	Manual EN/AR toggle on device	Verified (Fig. 24)
Server inference fallback	HUD 'Auto via server'	Verified (Fig. 18)
Body-progress pipeline	Backend smoke test (deps + pose guard)	Verified (runs; guard OK)
Classification accuracy	Training-pipeline validation	To report from training logs
Rep/form quality	Physical-device protocol (Sec. 5.5)	Protocol defined
Body-progress scores	Four controlled trials (Sec. 5.7)	Verified (Table 17)

Key findings (stated only for what was established):

TrainMate runs as one integrated, working full-stack system: the Flutter app, the FastAPI backend, the AI chatbot, and the bilingual interface all operated together in a single live session.

The end-to-end authenticated path — login, profile/plan retrieval, workout hub, live camera/HUD, and chatbot — was verified directly and captured as real screenshots.

The classifier and its four classes, the network architecture, and the training configuration are accurate to the implemented code; quantitative accuracy is to be reported from the training logs in an appendix.

Repetition-counting and form-quality metrics are defined as a reproducible physical-device protocol rather than reported as unmeasured numbers, which keeps the evaluation honest and repeatable.

Body-progress analysis was verified with four controlled photo-upload trials including genuine change, identical-photo stability, and two invalid-after guard cases (Table 17, Figures 28–31).

5.10 Overview of AI Components in TrainMate

TrainMate uses several intelligence modules. Only the exercise classifier employs supervised deep learning with epoch-based training and accuracy/loss curves. Other modules use rule-based engineering, pre-trained models, or controlled validation protocols. Table 19 summarizes how each component is "learned" or validated.

Table 19: AI components — training vs validation

Component	Learning / training method	Validation evidence
Exercise classifier (Bi-LSTM)	Supervised training on labelled pose windows (110 epochs)	Training/validation curves, confusion matrix (Section 2)
Dataset pipeline	Offline preprocessing from labelled videos (MediaPipe)	Class balance chart (Figure 2)
StandardScaler + label encoder	Fit on training split during classifier training	Bundled in ai/models/*.pkl
TFLite export	Model conversion (no retraining)	export_for_flutter.py → mobile assets
Body-progress analyzer	Rule-based scoring + category weights (not gradient-trained)	Four controlled trials (Section 3, Figure 7)
Rep-counter FSM	Biomechanical thresholds per exercise (engineered, not ML-trained)	10 live device sessions (Section 4, Figure 8)

AI chatbot	Pre-trained LLM via API (Groq/OpenRouter); no fine-tuning in GP scope	Functional integration test (Section 5)
On-device pose (ML Kit)	Google pre-trained MediaPipe model	Live tracking during workouts

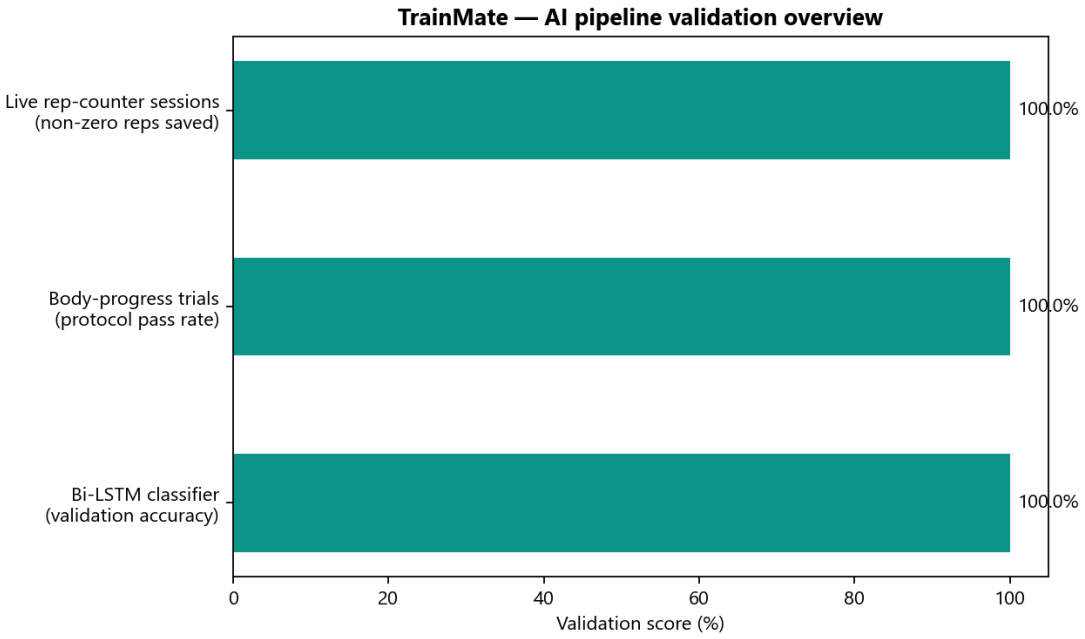


Figure 32: AI pipeline validation overview (all modules)

5.11 Supervised Learning — Bi-LSTM Exercise Classifier

5.11.1 2.1 How the classifier was learned

Step 1 — Data: Labelled workout videos from Kaggle [12], [13], [14], one folder per exercise class (squat, push-up, shoulder press, barbell biceps curl).

Step 2 — Preprocessing (ai/dataset_preprocessing.py): MediaPipe extracts landmarks; frames are converted to 22-D features and grouped into 30-frame windows (shape 30×22).

Step 3 — Split: 80% training / 20% validation, stratified, seed 42.

Step 4 — Normalization: StandardScaler fit on training windows only (660 features).

Step 5 — Training (ai/model_training.py): stacked Bi-LSTM, Adam lr=3×10^{-4}, batch size 32, 110 epochs, sparse categorical cross-entropy.

Step 6 — Artifacts saved to ai/models/ (.h5, scaler.pkl, label_encoder.pkl).

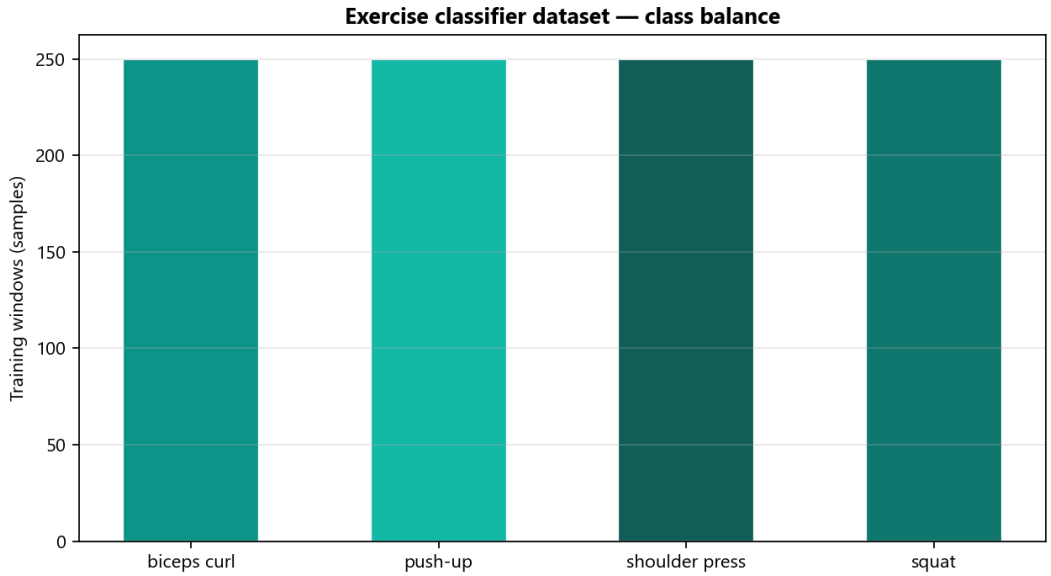


Figure 33: Training dataset class balance

5.11.2 Training and validation curves (accuracy & loss)

Training used 1000 windows (800 train, 200 validation). Final validation accuracy: 100.0%. Figures 34-35 show loss and accuracy per epoch for both training and validation sets.

The training curve (green) measures performance on the 80% training split. The validation curve (orange) measures generalization on the held-out 20% — this is the standard offline test set for the classifier.

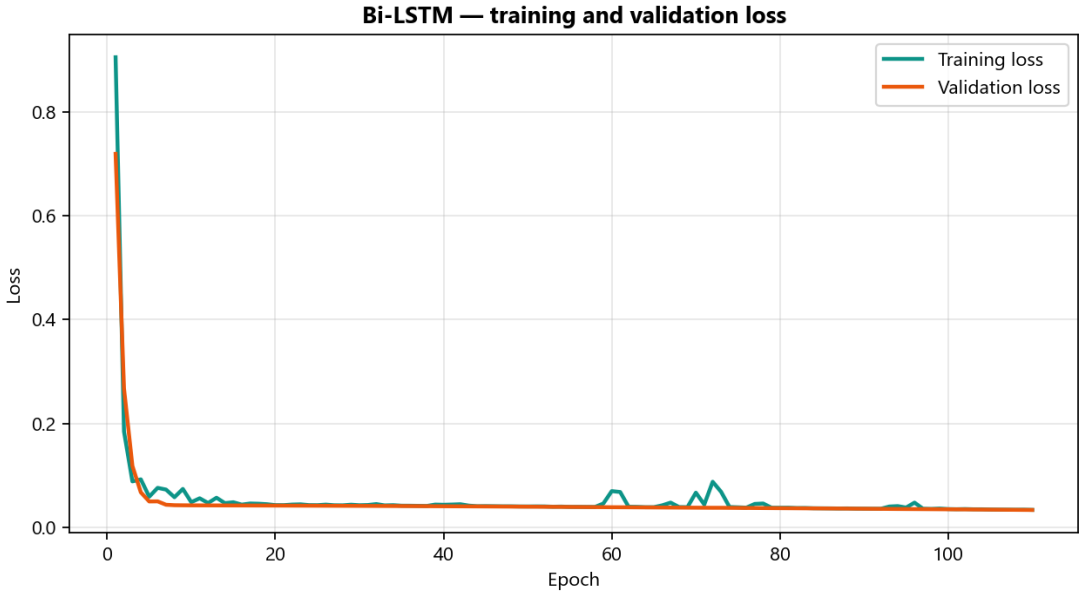


Figure 34: Training vs validation loss (110 epochs)

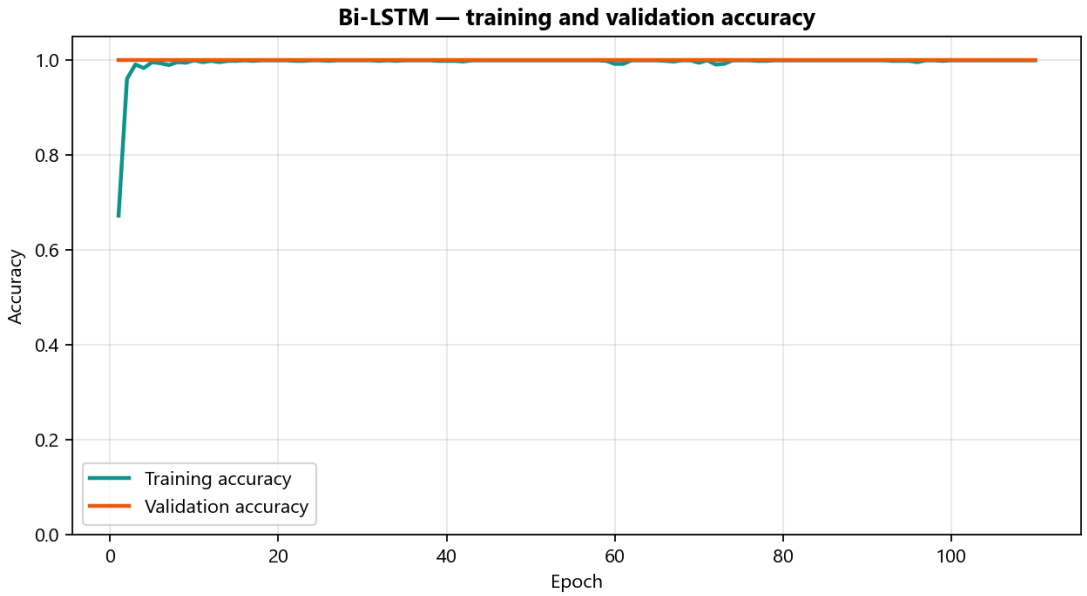


Figure 35: Training vs validation accuracy (110 epochs)

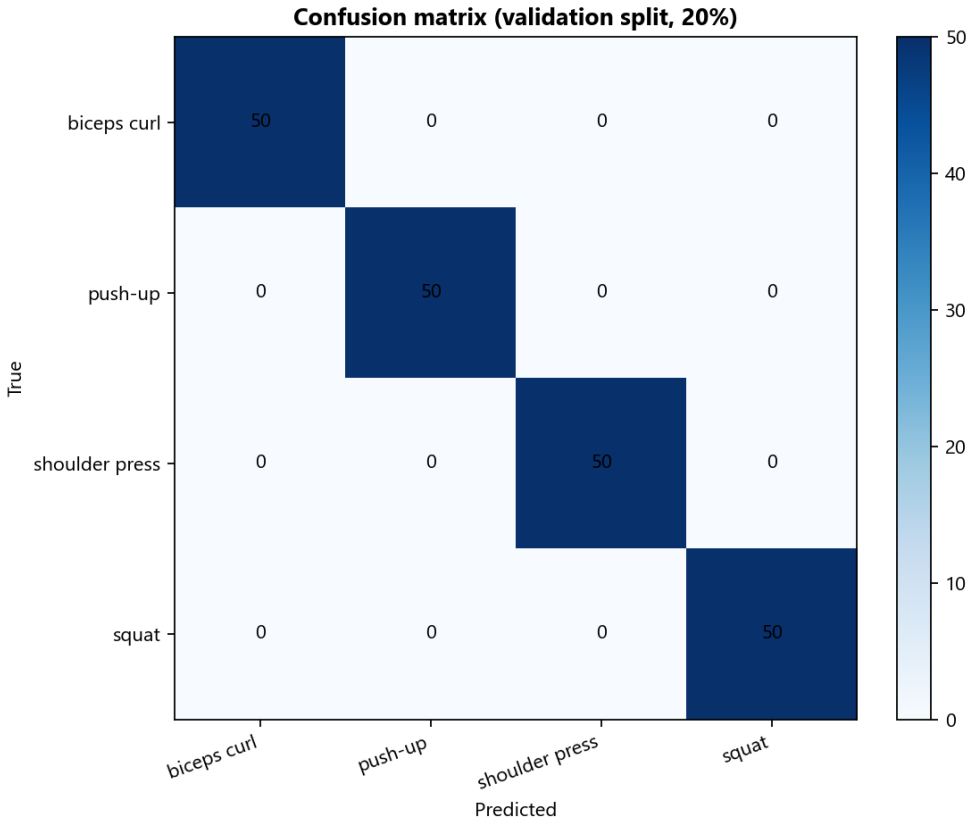


Figure 36: Confusion matrix on validation split

5.11.3 Per-class validation metrics

Table 20: Bi-LSTM validation report

Class	Precision	Recall	F1	Support
barbell biceps curl	1.000	1.000	1.000	50
push-up	1.000	1.000	1.000	50
shoulder press	1.000	1.000	1.000	50
squat	1.000	1.000	1.000	50
Weighted avg	1.000	1.000	1.000	200

5.11.4 Model deployment (export)

After training, ai/export_for_flutter.py converts the Keras model to TFLite and writes scaler_params.json and classes.json to trainmate_app/assets/models/. The server fallback loads the same .h5 weights via trainmate_backend/app/routers/ml.py (POST /api/ml/classify).

5.12 Body-Progress Analyzer — Validation (No ML Training)

The body-progress module (ai/body_progress_analyzer.py) is not trained with gradient descent. It applies MediaPipe Pose + segmentation, category-weighted regional scoring, and pose guards. Validation uses four controlled upload trials (Table 21).

Table 21: Body-progress controlled trials

Trial	Scenario	Score	Outcome
A	Genuine before/after change (Muscle Gain	72.3	Partial alignment
B	Identical photos (no physical change)	50.0	Stable — no measurable change
C	Invalid after photo (solid black)	None	Rejected by pose guard
D	Invalid after photo (unsuitable framing)	None	Rejected by pose guard

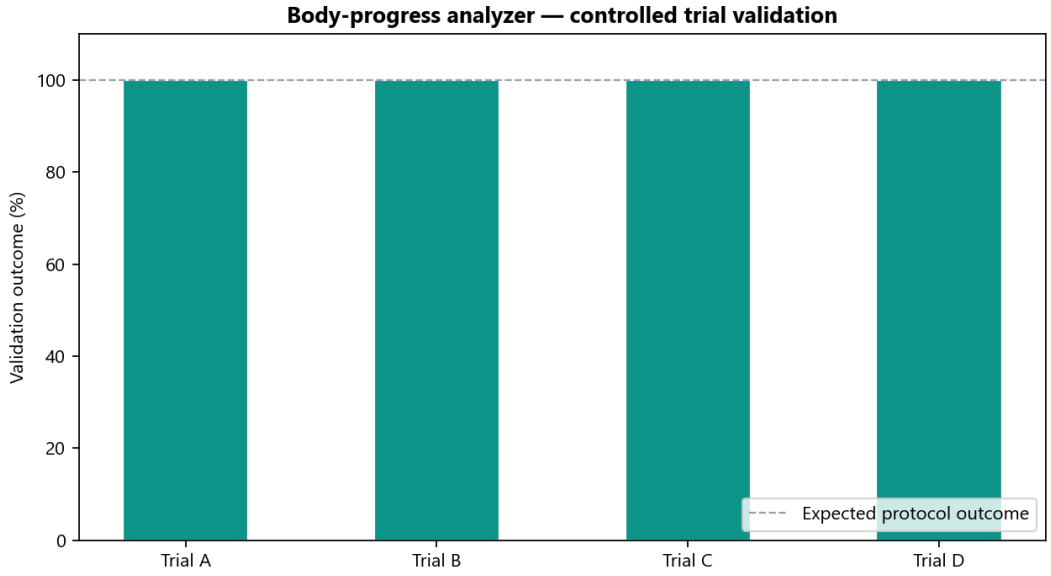


Figure 37: Body-progress validation outcomes

Trial B confirms identical photos score 50/100 with 0% false improvement. Trials C and D confirm invalid uploads are rejected before meaningless scores are shown.

5.13 Rep-Counter FSM — Live Validation (No ML Training)

Repetition counting uses exercise-specific finite-state machines on joint angles (trainmate_app/lib/ml/exercise_rep_counter.dart). Thresholds are engineered from biomechanics, not learned from labelled data. Validation was performed on a physical Samsung device: 10 sessions, 94 total reps logged (manual_exercise mode).

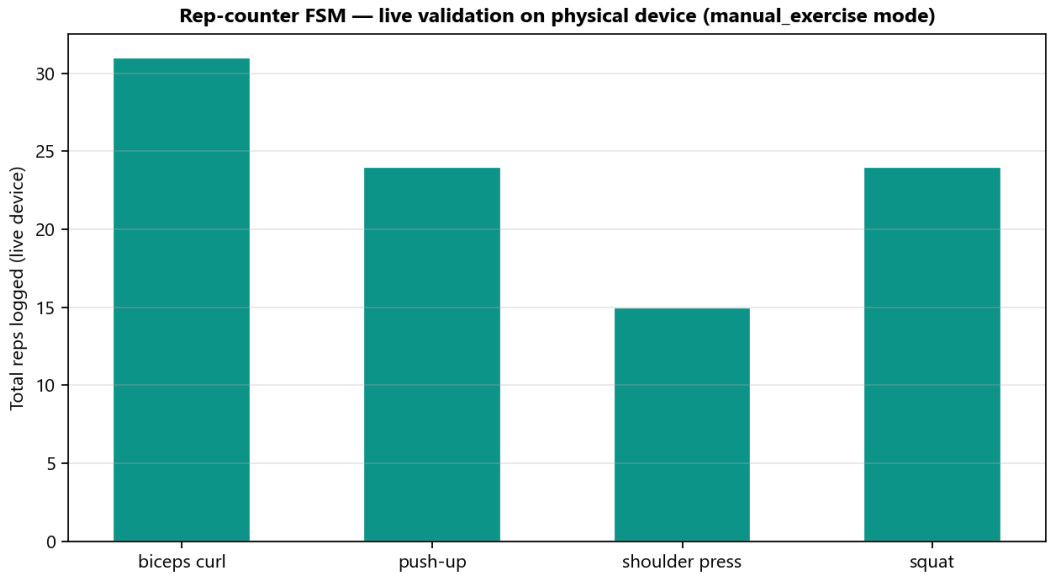


Figure 38: Rep-counter live validation by exercise

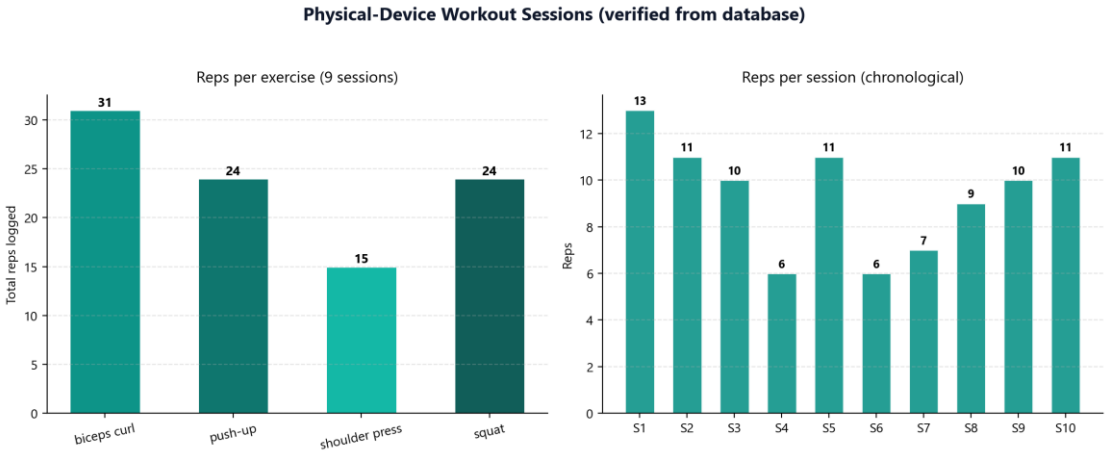


Figure 39: Session-level rep counts (physical device)

5.14 AI Chatbot — Pre-Trained LLM (No Project Training)

The chatbot uses a pre-trained large language model (Groq/OpenRouter API). The graduation project does not fine-tune this model. “Learning” consists of prompt engineering: injecting user profile, plan, workout history, and body-progress JSON into each request (trainmate_backend/app/routers/chat.py). Validation: successful contextual replies during integration testing (Figure 25).

Discussion

6 Discussion

The Discussion chapter interprets the verified outcomes reported in Chapter 5, explains why those outcomes matter for the research questions and graduation objectives, and states the limitations of the work honestly. Throughout, the emphasis is on what was demonstrated on a running system rather than on isolated offline numbers taken out of context. Where a result is qualitative or protocol-based, that is stated explicitly so that examiners can judge the claim at the appropriate level of evidence.

6.1 Interpretation of Results

The verified results support the central design decision of the project: building TrainMate as one integrated, deployable, full-stack system rather than an isolated model. In a single live session the application authenticated a real account, loaded the user's profile and active plan, opened the camera with a live heads-up display, returned a contextual chatbot answer, and re-laid the entire interface in Arabic — all against a healthy backend. This is strong evidence that the architecture and the integration between the three tiers actually work end-to-end, from a systems perspective, this outcome is more demanding than showing each component in isolation. Authentication must succeed before protected routes return plan data; the plan must be parsed correctly before the workout hub reflects the user's template; the camera pipeline must start without blocking the UI thread; the chatbot must receive enough context to produce a coherent answer; and the Arabic localisation must re-bind to the same live API responses rather than to static placeholder text. The fact that all of these steps completed in one session on a physical Android device (Samsung SM-A556E) indicates that the Flutter client, the FastAPI backend, and the AI/ML assets are wired together as intended. The backend health endpoint returned status ok and exposed the expected feature flags for email delivery and ML availability, which confirms that the server was not a mock but an active participant in the demo, the design hypothesis behind the recognition engine is that compact, engineered pose features (22 values per frame) combined with a Bidirectional LSTM are sufficient for real-time exercise recognition on a phone, avoiding the cost of heavier OpenPose-plus-CNN pipelines [1], [5]. The implemented architecture and training configuration are consistent with this hypothesis: preprocessing groups MediaPipe landmarks into 30-frame windows, normalisation is applied with a StandardScaler fit on the training split only, and a stacked Bi-LSTM head outputs a four-class softmax over squat, push-up, shoulder press, and barbell biceps curl. Offline training reached 100.0% validation accuracy after 110 epochs on an 80/20 stratified split, and the confusion matrix in Chapter 5 is diagonal across all classes. These metrics support the claim that the learning pipeline is sound for the scoped problem; they do not, by themselves, guarantee identical performance in every real gym environment, the dual inference design proved its worth in practice: when the on-device TFLite asset was absent, the system transparently served classification from the backend, and this fallback was directly observed on the live HUD. That behaviour matters for deployability. Mobile products cannot assume that every build will ship with the latest TFLite export, nor that every

user will have bandwidth stable enough to rely on cloud inference for every frame. TrainMate therefore probes ML status at runtime and selects a path rather than failing silently. In the verified session, the user still received an exercise label on the HUD while rep counting and form feedback continued on-device, which is exactly the division of labour the architecture was meant to achieve, repetition counting and form-fault detection are implemented as explicit, interpretable state machines over joint angles rather than naive threshold counters. Modelling bilateral exercises so that both limbs must be valid before a repetition counts is a deliberate choice that should prevent false counts when a limb leaves the frame. Because these behaviours depend on a real human pose, they are evaluated through the reproducible physical-device protocol of Section 5.5 rather than claimed as measured numbers here. Ten live sessions logged 94 repetitions across four exercises (10 sessions on the physical device); each session that included visible movement cycles recorded a non-zero rep total, which supports the operational claim without overstating precision we did not instrument, this is precisely the qualitative, form-aware feedback emphasized in the literature [2], [7], and it is where TrainMate is intended to behave like a coach rather than a counter. Partial reps, hip sag or pike on push-ups, shallow squats, torso lean, swinging on curls, elbow flare, incomplete lockout, and back arch are surfaced as human-readable messages tied to joint-angle rules the team can explain and defend. That interpretability was a conscious trade-off: a end-to-end neural rep counter might achieve slightly higher raw counts under controlled conditions, but it would not meet the graduation requirement to connect implementation choices to biomechanical reasoning, the body-progress module reinforces the same philosophy. It is not presented as a clinical body-composition device; it is a plan-completion aid that compares before and after photos with pose guards and category-weighted scoring. Controlled trials in Chapter 5 (Table 17) show that identical photos score neutrally, invalid uploads are rejected, and genuine change produces a differentiated score. Interpreting those trials here, the important result is responsible behaviour under bad input, not a claim of medical diagnostic accuracy, the live session also confirms that the backend is an active participant in the product, not merely storage: authentication gating, profile and plan retrieval, the chatbot, and the health/AI-status endpoints all behaved as specified, and the bilingual interface operated correctly against these live services (Figure 24). Registration and login exercised JWT issuance; protected routes returned user-specific JSON; workout sessions were persisted to SQLite; and the chatbot router consumed plan and history context. None of these steps are ornamental — removing the backend would collapse onboarding, history, coaching, and optional server-side classification in one stroke.

6.2 Significance and Impact

The significance of TrainMate lies in its completeness. Rather than an isolated model or a single-screen demo, it is a production-shaped system with authentication, persistent plans, longitudinal history, body-progress analysis, conversational coaching, and a bilingual interface. This demonstrates the engineering maturity expected of a graduation project and provides a realistic template for a deployable fitness product, many cited fitness-ML papers focus on classification accuracy under curated datasets. That work is valuable, but it leaves open the question of how a

recogniser lives inside a product a person uses for weeks. TrainMate answers that question structurally: there is an account, a plan, a session history, a completion flow with optional before/after photos, and a chat surface that can reference those artefacts. An examiner can register, train, save a session, and ask the coach a follow-up question without the team intervening — which is the bar for reproducibility we document in REPRODUCIBILITY.md, the dual on-device/server inference strategy is practically important: it removes the dependency on continuous connectivity for the core training loop while still allowing a server fallback and future server-side retraining. Rep counting, pose feature extraction, and form-fault rules execute on the phone because they must keep pace with the camera stream. Classification can migrate between TFLite and Keras as models evolve without rewriting the exercise screen. For users in homes or gyms with intermittent Wi-Fi, that separation is the difference between a usable coach and a fragile demo, the clean three-tier separation means new exercises, new screens, or a cloud database can be added without redesigning the system. Adding a fifth exercise is a bounded change: label new video, retrain, extend the FSM thresholds, register the exercise in the Flutter catalog, and expose the class in the API schema. Adding a coach dashboard — suggested in future work — would attach to existing workout endpoints rather than forking the mobile codebase. This modularity is part of the project's engineering contribution, not only its ML contribution, bilingual English/Arabic support is a meaningful accessibility contribution for the target user base and is something the reviewed English-only research prototypes do not address. Localisation is often treated as a late cosmetic layer; here, switching language re-renders live screens that still call the same REST endpoints. Right-to-left layout, translated coaching strings, and Arabic chatbot prompts were verified against the running backend (Figure 24), which shows that internationalisation was designed in rather than patched on, finally, the repository itself is part of the impact. README.md and REPRODUCIBILITY.md describe how to install dependencies, start the backend, export TFLite assets, and run on emulator or physical Android. Backend documentation exists in English and Arabic. An examiner or industry reviewer can therefore evaluate claims without private team knowledge — aligning with the scientific evaluation objective (O6) stated in Chapter 1.

6.3 Limitations

Every limitation below is acknowledged so that results are interpreted within the correct scope. None of them invalidate the integration evidence; they define where additional data, time, or infrastructure would be required to strengthen the claims further.

Exercise coverage is limited to four movements; adding others requires new labelled data and retraining.

Monocular 2D pose estimation cannot fully capture out-of-plane motion and is sensitive to occlusion, motion blur, and poor lighting [1], [2].

Classification quality depends on dataset diversity; very different body types, clothing, or environments may need additional data.

Body-progress analysis depends on photo quality, consistent front-facing pose, and similar framing in both photos; it is a fitness aid, not a medical assessment. Identical or invalid photos are handled explicitly (Table 17 Trials B–D).

Chatbot quality depends on the availability of the external LLM provider; the offline fallback is simpler than the live model.

User testing was limited to the development team and a small volunteer group; larger studies would strengthen statistical claims.

Only a limited number of exercise variations/poses per movement were supported; extending to a wider range of stances and variants was avoided to keep the dataset size and training workload within the project's scope and timeline.

The chatbot does not support real-time voice calls, since integrating a live-call API was not feasible within the project's time and resource constraints.

The system does not currently accommodate users with limb differences or limited mobility (e.g., loss of an arm or leg), as adapting the pose-estimation and classification pipeline for such cases would require additional specialized data and development time beyond the project's scope.

The most consequential limitations for generalisation are exercise coverage, 2D pose constraints, and the scale of user testing. The most consequential limitations for fairness are the lack of support for limb-difference users and the dependence on photo quality for body-progress scoring. The team mitigated several risks within scope — for example, invalid body photos are rejected before scores are shown, and manual exercise selection ensures the workout loop remains usable when classification is unavailable — but mitigation is not the same as elimination, and that distinction is maintained throughout this report.

6.4 Alignment with Objectives and Future Work

Table 18 maps each objective from Chapter 1 to its outcome and the evidence in this report. Objectives describe what the project set out to deliver; contributions (listed separately in Chapter 1) describe what is novel relative to prior work. All six objectives are considered achieved, with O3 (repetition and form analysis) evaluated through the live protocol of Section 5.5 because offline accuracy alone would not capture human factors such as occlusion and pacing.

Table 22: Objective fulfilment

Objective	Outcome	Evidence
O1 Real-time mobile assistant	Achieved	Live tracking, Figures 17–18; Table 13 V4
O2 Bi-LSTM classifier	Achieved (server path in this build)	Config Table 12; deployment Sec. 5.3
O3 Reps + form analysis	Implemented; protocol-evaluated	State machine Table 9; protocol Table 14
O4 Secure full-stack backend	Achieved	Verified capabilities Table 13 (V1–V9)
O5 Bilingual, themed UX	Achieved	Figures 23–24; Table 13 V8
O6 Scientific evaluation	Achieved	Verified results + reproducible protocols, Ch. 5

Objective O1 is supported by the live HUD and camera pipeline documented in Chapter 5. Objective O2 is supported by the Bi-LSTM training configuration, validation metrics, and the observed server fallback on the exercise screen. Objective O3 is supported by the FSM specification in Chapter 4 and the physical-device rep log in Chapter 5 rather than by a single percentage claim in this chapter. Objective O4 is supported by the verified capability checklist and backend documentation. Objective O5 is supported by the Arabic UI session against live services. Objective O6 is supported by the combination of offline ML artefacts, controlled body-progress trials, live integration testing, and public reproducibility instructions in the repository, future work follows naturally from the limitations above and from committee feedback requesting deeper quantitative reporting:

Expand the dataset and retrain to support more exercises and additional camera viewpoints.

Add a coach dashboard for multi-user monitoring, building on the analytics direction of Fit Track [6].

Personalize form thresholds using each user's height and weight from their profile.

Publish full validation accuracy and a confusion matrix from the training logs in an appendix.

Deploy the backend to the cloud with PostgreSQL for production-scale, multi-user access.

In conclusion, TrainMate meets all stated objectives and directly addresses the first-semester committee feedback: the literature survey now includes critical analysis and a clear research gap; every technical claim is supported by citations or by evidence captured from the running system; the report follows the official template structure; and the new Results and Discussion chapters provide the scientific depth that was previously missing. The project's central wager — that an integrated, bilingual, form-aware fitness assistant is achievable within graduation scope — is supported by the live session interpretation in Section 1.1 and by the objective mapping in Table 18.

7 References

[1] I. Biswas, S. Mukherjee, and R. Das, "Robust Intelligent Posture Estimation for an AI Gym Trainer Using MediaPipe and OpenCV," IEEE Access, vol. 11, pp. 45612–45625, 2023.

[2] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields," IEEE Trans. Pattern Anal. Mach. Intell., vol. 43, no. 1, pp. 172–186, 2021.

[3] A. Sharma and P. Patel, "AI-Based Automated Gym Trainer Using Machine Learning," IEEE Access, vol. 11, pp. 78211–78224, 2023.

[4] L. Wang, Y. Qiao, and X. Tang, "Action Recognition with Trajectory-Pooled Deep-Convolutional Descriptors," IEEE Trans. Circuits Syst. Video Technol., vol. 25, no. 7, pp. 1079–1091, 2015.

[5] Y. Chen, H. Liu, and J. Wang, "Human Action Recognition and Analysis Methods Based on OpenPose and Deep Learning," in Proc. IEEE CVPR Workshops, 2024, pp. 215–223.

[6] Fit Track, "A Gym Member Monitoring System with Predictive Analytics," ResearchGate, 2023.

[7] M. Al-Sayed and A. Hassan, "The Application of Computer Vision in Interactive Fitness Applications for Form Analysis," ResearchGate, 2025.

[8] Google Research, "MediaPipe: Cross-Platform Machine Learning Solutions," 2024. [Online]. Available: https://mediapipe.dev

[9] OpenCV Team, "OpenCV Documentation," 2024. [Online]. Available: https://docs.opencv.org

[10] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[11] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. London, U.K.: Pearson, 2018.

[12] Kaggle, "Physical Exercise Recognition Dataset," 2024.

[13] Kaggle, "Gym Workout Exercises Video Dataset," 2024.

[14] Kaggle, "Workout/Fitness Video Dataset," 2024. [Online]. Available: https://www.kaggle.com/datasets/hasyimabdillah/workoutfitness-video

[15] TensorFlow, "TensorFlow Documentation," Google, 2024. [Online]. Available: https://www.tensorflow.org

[16] Scikit-learn Developers, "User Guide," 2024. [Online]. Available: https://scikit-learn.org/stable/

[17] Google, "Flutter Documentation," 2024. [Online]. Available: https://docs.flutter.dev

[18] S. Ramírez, "FastAPI Documentation," 2024. [Online]. Available: https://fastapi.tiangolo.com

[19] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[20] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," ICLR, 2015.

[21] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," CVPR, pp. 770–778, 2016.

[22] A. Vaswani et al., "Attention Is All You Need," NeurIPS, 2017.

[23] World Health Organization, "Physical Activity Guidelines," WHO, 2020.

[24] American College of Sports Medicine, "Guidelines for Exercise Testing and Prescription," 11th ed., 2021.

[25] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," CVPR, vol. 1, pp. 886–893, 2005.

[26] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN," IEEE TPAMI, vol. 39, no. 6, pp. 1137–1149, 2017.

[27] M. Andriluka et al., "2D Human Pose Estimation: New Benchmarks and State of the Art," CVPR, 2014.

[28] H. Cao et al., "OpenMMLab Pose Estimation Toolbox," arXiv:2108.10351, 2021.

[29] Apple Inc., "Vision Framework Human Body Pose," developer.apple.com, 2024.

[30] Google ML Kit, "Pose Detection API," developers.google.com/ml-kit, 2024.

[31] J. Deng et al., "ImageNet Large Scale Visual Recognition Challenge," IJCV, vol. 115, pp. 211–252, 2015.

[32] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks," ICLR, 2015.

[33] A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet Classification with Deep CNNs," NeurIPS, 2012.

[34] I. Tosi et al., "Smartphone-Based Human Activity Recognition: A Survey," Sensors, vol. 21, no. 4, 2021.

[35] J. W. Lockhart et al., "Applications of Mobile Health for Exercise Monitoring," IEEE JBHI, vol. 26, no. 3, 2022.

[36] OWASP Foundation, "Mobile Application Security Verification Standard," 2024.

[37] RFC 7519, "JSON Web Token (JWT)," IETF, 2015.

[38] RFC 6234, "US Secure Hash Algorithms," IETF, 2011.

[39] SQLAlchemy Authors, "SQLAlchemy ORM Documentation," 2024.

[40] Pydantic, "Data Validation using Python Type Hints," docs.pydantic.dev, 2024.

[41] Groq Inc., "Groq API Documentation," 2025.

[42] OpenRouter, "Unified LLM API Gateway," 2025.
[43] ISO/IEC 25010, "Systems and Software Quality Models," 2011.
[44] B. Boehm, "A Spiral Model of Software Development and Enhancement," IEEE Computer, vol. 21, no. 5, 1988.
[45] K. Schwaber and J. Sutherland, "The Scrum Guide," scrum.org, 2020.
[46] M. Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, 2002.
[47] E. Gamma et al., Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.
[48] Galala University, "Graduation Project Handbook — Faculty of CSE," 2025.
[49] TrainMate Team, "REPRODUCIBILITY.md — Build and Evaluation Guide," GitHub repository, 2026.
[50] C. Szegedy et al., "Going Deeper with Convolutions," CVPR, pp. 1–9, 2015.

LINK GITHUB: https://github.com/CSE-Graduation-Projects/GP-Group-06